



UNIVERSIDADE ESTADUAL DE CAMPINAS
Faculdade de Tecnologia

Maria Eduarda Oliveira da Silva

**Uso de Grafos Acíclicos Direcionados em Camadas para
Visualização de Múltiplos Históricos Escolares**

Limeira
2025

Maria Eduarda Oliveira da Silva

**Uso de Grafos Acíclicos Direcionados em Camadas para Visualização
de Múltiplos Históricos Escolares**

Monografia apresentada à Faculdade de
Tecnologia da Universidade Estadual de Campinas
como parte dos requisitos para a obtenção do
título de Tecnólogo em Análise e
Desenvolvimento de Sistemas, na área de
Sistemas de Informação e Comunicação.

Orientador: Prof. Dr. Celmar Guimarães da Silva

Coorientador: Prof. Thiago Gonçalves Mendes

Este trabalho corresponde à versão final da
Monografia defendida por Maria Eduarda
Oliveira da Silva e orientada pelo Prof. Dr.
Celmar Guimarães da Silva.

Limeira
2025

Prof. Dr. Celmar Guimarães da Silva
Presidente da Comissão Julgadora

Prof.^a Dr.^a Ana Estela Antunes da Silva
FT/UNICAMP

Prof.^a Dr.^a Gisele Busichia Baioco
FT/UNICAMP

Ata da defesa, assinada pelos membros da Comissão Examinadora, encontra-se no SIGA/Sistema de Fluxo de Dissertação/Tese e na Secretaria de Graduação da Faculdade de Tecnologia.

Agradecimentos

Primeiramente, agradeço à minha família — aos meus pais, Cesar e Elisângela, e aos meus avós, Jurandir, Junquera e Maria — por acreditarem em mim, apoiarem meus sonhos e sempre colocarem minha educação como prioridade.

Agradeço ao meu orientador, Prof. Dr. Celmar Guimarães da Silva, e ao meu coorientador, Prof. Thiago Gonçalves Mendes, por toda a orientação, paciência e incentivo durante esta jornada, bem como por me introduzirem a uma área de pesquisa com a qual tanto me identifiquei e que levarei comigo para a vida. Estendo também minha gratidão à Faculdade de Tecnologia, por tornar possível o sonho da graduação e por todo o conhecimento proporcionado.

Aos meus amigos que em todos os momentos que precisei, estavam lá para me apoiar e me ajudaram nessa jornada. Ao meu namorado, Guilherme, por ter me acompanhado durante esse caminho, por ter tido toda a paciência e carinho que eu precisava e por sempre acreditar em mim.

Às minhas cachorras — as duas Cindy, Lily, Lolla e Luna — que me trouxeram e trazem alegria diariamente, mesmo à distância.

Agradeço às Embaixadoras da Ciência e Tecnologia, por me inspirarem e me motivarem a inspirar outras meninas e mulheres nas áreas de ciência, tecnologia, engenharia e matemática.

Agradeço às entidades e Orixás, que iluminaram meu caminho e me acompanham sempre me dando forças.

Expresso também minha gratidão a figuras e obras que contribuíram para minha formação pessoal. Agradeço àqueles cujas músicas, filmes e histórias, especialmente as da artista Lady Gaga, me acompanharam ao longo dos anos, principalmente por transmitirem mensagens de força, autenticidade e esperança. Suas criações moldaram parte de quem sou e me ensinaram a acreditar em mim, mesmo diante dos desafios.

E por fim, mas não menos importante, a todos os fandoms que faço e fui parte. Se tem uma coisa que me fez chegar até aqui foi minha paixão por arte, música, séries e filmes. Os fandoms sempre fizeram parte da minha vida, foram fontes de conforto, inspiração e conexão humana, influenciando minha forma de ver o mundo e motivando-me a continuar perseguindo meus objetivos com paixão e sensibilidade.

Resumo

Acompanhar o desempenho de alunos universitários em disciplinas de um curso exige elevado esforço cognitivo por parte das coordenações, uma vez que os dados acadêmicos são geralmente disponibilizados de forma predominantemente alfanumérica. Nesse contexto, técnicas de Visualização de Informação podem ampliar a compreensão de dados educacionais complexos, especialmente quando envolvem múltiplos históricos escolares. Este trabalho apresenta uma proposta de visualização baseada em grafos acíclicos direcionados em camadas, considerando as relações de pré-requisito entre disciplinas e a progressão temporal dos semestres, permitindo a análise visual simultânea de diversos históricos acadêmicos. A metodologia adotada foi exploratória e incremental, envolvendo levantamento de requisitos, definição de tarefas analíticas e desenvolvimento de um protótipo funcional, validado por meio de dados sintéticos gerados pelo sistema GradeGen. Os resultados indicam que a abordagem facilita a identificação de padrões de desempenho, gargalos curriculares e disciplinas críticas, além de possibilitar comparações visuais integradas entre históricos, contribuindo para a redução da sobrecarga cognitiva e para o apoio à tomada de decisão por coordenadores de curso e professores.

Abstract

Monitoring the performance of university students in course subjects requires significant cognitive effort from coordinators, since academic data is generally provided predominantly in alphanumeric format. In this context, Information Visualization techniques can enhance the understanding of complex educational data, especially when involving multiple academic records. This work presents a visualization proposal based on layered directed acyclic graphs, considering the prerequisite relationships between subjects and the temporal progression of semesters, allowing for the simultaneous visual analysis of various academic records. The methodology adopted was exploratory and incremental, involving requirements gathering, definition of analytical tasks, and development of a functional prototype, validated through synthetic data generated by the GradeGen system. The results indicate that the approach facilitates the identification of performance patterns, curricular bottlenecks, and critical subjects, in addition to enabling integrated visual comparisons between records, contributing to the reduction of cognitive overload and supporting decision-making by course coordinators and professors.

Lista de Figuras

3.1	CPN completo das disciplinas da Benedictine University	20
3.2	Subgrafo do curso de Bioquímica e Biologia Molecular na Benedictine University	21
3.3	Centralidade das disciplinas do curso de Bioquímica e Biologia Molecular . . .	22
3.4	Demo Curricular Analytics	24
3.5	Legenda da Disciplina no Curricular Analytics	25
3.6	Software CourseViewer	27
3.7	Currículo Exemplo no CourseViewer sem <i>edge bundling</i> e com curvas cardinais	28
3.8	Currículo Exemplo no COurseViewer com <i>edge bundling</i> e com curvas cardinais	28
3.9	Matriz reordenada e mapa de calor	31
4.1	Desenho do Fluxo das Etapas de Desenvolvimento	34
4.2	Exemplo de uma Simulação do GradeGen	38
4.3	Diagrama - Classe Disciplina	40
4.4	Código - Métodos da Classe Disciplina	41
4.5	Diagrama - Classe Catálogo	42
4.6	Código - Método do CSV na classe Catalogo	43
4.7	Exemplo de Funcionamento NetworkX	44
4.8	Código - Função para Criar Dígrafo	44
4.9	Código - Agrupamento de nós e Cálculo de Posições	45
4.10	Código - Configuração da tela de desenho	46
4.11	Código - Caixas personalizáveis dos semestres	46
4.12	Código - Vetores de direção para as setas	47
4.13	Código - Função de mapeamento de cor nas notas	48
4.14	Código - Função desenhar mini mapa de calor	49
4.15	Código - Desenhar caixas dos nós com mini mapa de calor	50
4.16	Diagrama - Classe App	51
4.17	Código - Início da Função Create Widgets	52
4.18	Código - Função para Gerar Visualização do Grafo	53
4.19	Código - Destacar caixas e textos ao passar do mouse	54
4.20	Visualização do sistema - Primeira Simulação SI	56
4.21	Destaque do Cursor do Mouse	57
4.22	Zoom in no nó da disciplina EB101	57
4.23	Barra Lateral de Informações	58
4.24	Informações da Disciplina Seleccionada	59
4.25	Visualização do curso de Engenharia Ambiental de 2023	60
5.1	Evidência de Gargalo no curso Engenharia Ambiental	62
5.2	Destaque da Disciplina EB901 em Engenharia Ambiental de 2023	63
5.3	Caminho da Disciplina EB402	64

5.4	Visualização do Curso Sistemas de Informações de 2026	65
5.5	Evidência da ordenação de RA	65

Lista de Tabelas

4.1	Tabela de tarefas do usuário	35
4.2	Tabela de Parâmetros de Alunos	36
4.3	Tabela de Fatores de Alteração	37

Lista de Abreviaturas e Siglas

BMB	Bioquímica e Biologia Molecular
CPN	<i>Curriculum Prerequisite Network</i>
DAGs	<i>Directed Acyclic Graphs</i>
EDM	<i>Educational Data Mining</i>
GUI	<i>Graphical User Interface</i>
IHC	Interface Humano-Computador
InfoVis	Visualização de Informação
LDAGs	<i>Layered Directed Acyclic Graphs</i>
OLO	<i>Optimal Leaf Order</i>
SEIS	<i>Software Engineering and Information Systems</i>
STEM	Ciência, Tecnologia, Engenharia e Matemática
UCSD	Universidade da Califórnia em San Diego
VLA	<i>Visual Learning Analytics</i>

Sumário

1	Introdução	12
2	Fundamentação Teórica	14
2.1	Visualização de Dados Educacionais	14
2.2	Grafos Acíclicos em Sistemas Educacionais	16
3	Levantamento e Análise de Trabalhos e Ferramentas Relacionados	19
3.1	Rede de Pré-Requisitos Curriculares	19
3.2	Software Curricular Analytics	23
3.3	CourseViewer	25
3.4	Uso de Matrizes Reordenáveis e Mapas de Calor para Análise de Históricos Escolares	29
4	Metodologia	33
4.1	Levantamento de Requisitos	34
4.2	Sistema GradeGen	36
4.3	Definição das técnicas de visualização	38
4.4	Bibliotecas Computacionais	39
4.5	Arquitetura do Software	40
4.5.1	Módulo model.py	40
4.5.2	Módulo viz.py	43
4.5.3	Módulo gui.py	50
4.5.4	Main.py	54
4.6	Proposta de Visualização	55
5	Resultados	61
6	Conclusões	67
	Referências bibliográficas	69

Capítulo 1

Introdução

Acompanhar o desempenho acadêmico de alunos universitários é uma atividade essencial para coordenadores de curso e professores. Essa tarefa envolve arquivos diversos de notas e estrutura curricular que, na maioria das vezes, não são integrados em um sistema único ou em uma visualização única.

Alguns trabalhos desenvolvidos pelo Grupo de Engenharia de Informação e Sistemas (FT/UNICAMP) têm explorado o uso de técnicas de Visualização de Informação (InfoVis) para representar currículos acadêmicos e históricos escolares. Em particular, grafos acíclicos direcionados em camadas já foram utilizados pelo grupo para a exibição de um histórico escolar por vez, por meio do software CourseViewer (Silva; Inoue; Mendonça, 2012). Nessa representação, os nós do grafo correspondem às disciplinas, as arestas direcionadas representam as relações de pré-requisito entre elas, e as camadas correspondem aos semestres letivos, enfatizando o fluxo temporal e a progressão acadêmica.

Porém, a análise simultânea de múltiplos históricos representa um desafio técnico e conceitual significativo. Esse desafio reside principalmente na escalabilidade visual: enquanto um único histórico pode ser representado claramente em um grafo, a sobreposição de múltiplas trajetórias individuais introduz uma complexidade que pode comprometer a legibilidade da visualização.

Diante desse cenário, o presente trabalho tem como objetivo propor e implementar visualizações baseadas em grafos acíclicos direcionados em camadas que ampliem, do ponto de vista técnico, as possibilidades de representação de históricos escolares e que, do ponto de vista analítico, apoiem a análise do desempenho acadêmico de múltiplos alunos

simultaneamente, preservando as relações de pré-requisito entre disciplinas e a progressão temporal dos semestres.

Este trabalho está organizado da seguinte forma. No Capítulo 2 são apresentados os fundamentos teóricos relacionados à visualização de dados educacionais e ao uso de grafos acíclicos em sistemas educacionais. No Capítulo 3 é realizado um levantamento e análise de trabalhos e ferramentas relacionados, incluindo o conceito de Curriculum Prerequisite Network, os software Curricular Analytics e CourseViewer, e o uso de matrizes reordenáveis e mapas de calor para a análise de históricos escolares. No Capítulo 4 é descrita a metodologia adotada, incluindo o levantamento de requisitos, o sistema GradeGen, a definição das técnicas de visualização, as bibliotecas computacionais utilizadas, a proposta de visualização desenvolvida e os resultados. Por fim, no Capítulo 5 são apresentadas as conclusões do trabalho.

Espera-se que a proposta e a implementação apresentadas neste trabalho possibilitem uma melhor compreensão dos dados de históricos acadêmicos e catálogos de cursos, auxiliando coordenadores de curso e professores na identificação de padrões de desempenho, disciplinas críticas e trajetórias acadêmicas dos estudantes, contribuindo assim para a melhoria dos processos educacionais e a redução do tempo de integralização dos cursos.

Capítulo 2

Fundamentação Teórica

Este capítulo apresenta os fundamentos teóricos que embasam o desenvolvimento da implementação proposta. A Seção 2.1 aborda os princípios da visualização da informação no campo educacional, e a Seção 2.2 explora grafos acíclicos direcionados utilizados como um sistema educacional, representando pré-requisitos e dependências.

2.1 Visualização de Dados Educacionais

De acordo com Card, Mackinlay e Shneiderman (1999), a visualização de informação pode ser definida como "o uso de representações visuais, apoiadas por computador, de dados abstratos, para amplificar a cognição". Essa definição ressalta o caráter instrumental da visualização, que não se limita à estética, mas atua como uma ferramenta de raciocínio analítico.

No âmbito educacional, essa abordagem dialoga diretamente com áreas emergentes que buscam compreender melhor os processos de aprendizagem a partir de dados. Um exemplo é a mineração de dados educacionais (*Educational Data Mining* – EDM), que, segundo Romero e Ventura (2010), consiste no desenvolvimento e na aplicação de métodos computacionais para explorar grandes volumes de dados provenientes de ambientes educacionais, com o objetivo de compreender o comportamento e o desempenho dos estudantes, bem como os contextos em que aprendem.

Essa área combina técnicas de mineração de dados, aprendizado de máquina e estatística para identificar padrões e dados significativos para o apoio e a melhoria dos processos de ensino e aprendizagem. Entre suas aplicações mais comuns estão o estudo do suporte pedagógico fornecido por softwares de aprendizagem, aprimoramento de modelos de

detecção e previsão de desempenho acadêmico. Assim, a EDM representa um ponto entre a análise automatizada e a compreensão pedagógica, oferecendo subsídios quantitativos para apoiar tomadas de decisões educacionais (Romero; Ventura, 2010).

Para tornar as descobertas da EDM compreensíveis, surgiram os *learning dashboard*. Em uma revisão sistemática sobre o tema, Schwendimann et al. (2017) definiram um *learning dashboard* como uma única tela que agrega diferentes indicadores sobre os alunos, seus processos e contextos de aprendizagem em uma ou múltiplas visualizações, com o objetivo de permitir a percepção do aprendizado "de relance" (*at a glance*). Com base nesse conceito, consolidou-se a área de análise visual da aprendizagem (*Visual Learning Analytics* – VLA), que, segundo uma revisão de escopo de Mohseni et al. (2024), combina a análise visual com a análise automatizada para fornecer explicações sobre os dados educacionais e, assim, auxiliar professores na tomada de decisões pedagógicas.

Esses desenvolvimentos evidenciam a relevância da visualização de dados educacionais em um cenário onde os registros acadêmicos, como históricos escolares, são frequentemente disponibilizados em formatos alfanuméricos, como em planilhas ou sistemas de gestão. Além disso, por vezes, não estão centralizadas em um único lugar. O grande volume também dificulta a análise desses dados como, por exemplo, a identificação de padrões, gargalos e trajetórias acadêmicas dos alunos. Nesse sentido, técnicas de visualização podem reduzir a sobrecarga das coordenações de curso, transformando informações complexas e dispersas em representações gráficas mais intuitivas.

Entre as principais vantagens das técnicas de InfoVis destacam-se: (i) a possibilidade de identificar relações não triviais entre elementos; (ii) a facilidade de detectar tendências ou anomalias; e (iii) o suporte a processos de tomada de decisão baseados em evidências visuais. A visualização de informação é fundamentalmente definida como sistemas baseados em computador que fornecem representações visuais de conjuntos de dados projetados para ajudar as pessoas a realizarem tarefas de forma eficaz. O objetivo central é aumentar as capacidades humanas e não substituí-las por métodos computacionais de tomada de decisão. Por isso, InfoVis é crucial quando se precisa ver a estrutura do conjunto de dados complexos, permitindo identificar elementos de interesse dos dados, compará-los entre si e obter uma visão geral do todo (Munzner, 2014).

Assim, a visualização de históricos escolares pode se beneficiar diretamente dessa abordagem, uma vez que a trajetória dos alunos pelas disciplinas de um catálogo constitui

um sistema complexo, permeado por dependências, possíveis reprovações e diferenças de caminhos.

No escopo deste trabalho, a visualização baseada em grafos acíclicos direcionados (*Directed Acyclic Graphs* – DAGs), mais especificamente os DAGs organizados em camadas (chamados de *Layered Directed Acyclic Graphs* – LDAGs). Esse recurso também será utilizado para representar não só o histórico de um aluno, mas permitir comparações entre múltiplos históricos simultaneamente.

2.2 Grafos Acíclicos em Sistemas Educacionais

Os grafos acíclicos direcionados constituem uma estrutura matemática que possibilita representar relações causais e de dependências, ou seja, nos quais os eventos ocorrem em uma ordem específica ou em que alguns nós afetam outros nós, mas os efeitos causais não necessariamente funcionam na direção oposta (IBM, 2025). Sendo assim, estruturas curriculares são inerentemente relacionais e se assemelham muito a grafos. Disciplinas são representadas pelos nós, enquanto as arestas direcionadas representam as relações de pré-requisito entre elas.

Um dos trabalhos pioneiros que exploraram o uso de DAGs em sistemas educacionais é o de Aldrich (2015) que introduziu o conceito de *Curriculum Prerequisite Network* (CPN), que utiliza um grafo para modelar o currículo de um curso, e as dependências entre disciplinas. Essa modelagem, junto com a teoria de grafos, permitiu analisar algumas características relevantes de um curso:

- Grau de um nó: A soma da quantidade das arestas de entrada e dos arcos de saída, se um nó tem um alto grau ele é considerado um "hub", o que indica uma centralidade estrutural de uma disciplina dentro de um catálogo. O grau ponderado pode ser avaliado como na equação 2.1:

$$k_i = \sum_{j=1}^N a_{ij} w_{ij} \quad (2.1)$$

Onde k_i é o grau ponderado do nó i , e $a_{ij} = 1$ se existir uma conexão entre os nós i e j , e $a_{ij} = 0$ caso contrário. A força da conexão é ajustada pelo peso w_{ij} , que expressa a intensidade ou importância da relação, esse peso pode indicar a força da dependência

entre as disciplinas (pré-requisito obrigatório ou opcional). Quando os pesos são 1.0 para todas as conexões não existe diferença de intensidade nas relações.

- Caminhos de integralização: Identificar o caminho mais curto de um nó para o outro, considerando que quanto mais nós em um caminho, maior a probabilidade de um aluno esquecer o conteúdo aprendido na disciplina do nó anterior
- Centralidade de intermediação: Índice que mede a frequência com que um nó aparece nos caminhos mais curtos entre pares de outros nós do grafo. quanto maior o valor de intermediação de uma disciplina, maior é sua importância na conectividade da rede curricular, pois ela atua como um tipo de "ponte".

A propriedade acíclica dos DAGs é particularmente relevante em currículos acadêmicos, pois garante que não existam dependências circulares entre disciplinas. Como observado por Aldrich (2015), a ausência de ciclos em um currículo é uma condição necessária para a progressão acadêmica ordenada, evitando situações onde uma disciplina A é pré-requisito de B, B é pré-requisito de C e C é pré-requisito de A, criando uma dependência impossível de ser satisfeita.

Neste trabalho, também, será adotada uma estrutura adicional ao organizar os nós em camadas, formando um grafo acíclico direcionado em camadas (LDAGs). Essa decisão é fundamentada na abordagem de desenho hierárquico de grafos, popularizada por Sugiyama, Tagawa e Toda (1981), cujo principal objetivo é otimizar a legibilidade de grafos direcionados. As camadas correspondem aos semestres dos anos acadêmicos, enfatizando o fluxo e a progressão temporal da estrutura.

Dessa forma, a adoção de DAGs em sistemas educacionais se mostra uma estratégia eficaz para lidar com a complexidade do processo de formação acadêmica. Ao possibilitar representações visuais de dados acadêmicos dos estudantes e dependências entre disciplinas, esses grafos fornecem contribuições valiosas para que coordenadores de curso e professores compreendam melhor os desafios e lacunas da progressão acadêmica, permitindo revelar padrões de fluxo de estudantes e outras informações que impactam diretamente no desempenho e no tempo de integralização (Raji et al., 2017).

Por sua vez, a análise simultânea de múltiplos históricos escolares representa um desafio técnico e conceitual significativo, pois envolve o agrupamento de trajetórias individuais distintas, mantendo a legibilidade das relações estruturais do currículo. Esta abordagem

difere fundamentalmente dos sistemas tradicionais que focam na análise de históricos individuais ou métricas agregadas de desempenho.

O principal desafio na visualização de múltiplos históricos reside na escalabilidade visual. Enquanto um único histórico pode ser representado claramente em um LDAG, o conjunto de trajetórias introduz uma complexidade que pode comprometer a eficácia da visualização. Esse problema, conhecido como desordem visual (*visual clutter*), ocorre quando a densidade de elementos gráficos (nós de disciplinas, arestas de pré-requisitos e notas) aumenta a ponto de degradar a capacidade do usuário de perceber padrões e extrair informações úteis (Ellis; Dix, 2006).

Essa sobrecarga informacional aumenta a carga cognitiva do usuário, exigindo mais esforço mental para interpretar o grafo, violando assim heurísticas de usabilidade criadas por Jakob Nielsen, que são diretrizes ou regras gerais de bom senso para a experiência do usuário com sistemas, como a diminuição do esforço cognitivo (Heurística 5 - Prevenção de Erros) e a diminuição do esforço visual (Heurística 4 - Consistência e Padronização), que recomendam evitar a sobrecarga visual, impedir que os usuários tenham que memorizar muitas informações e alinhar a visualização com o conhecimento e experiência deles (Morrone et al., 2023). Portanto, a proposta deste trabalho não se limita a agregar mais dados a uma estrutura existente, mas a integrar métodos de visualização e interação que permitam gerenciar essa complexidade, garantindo que a análise comparativa de múltiplos históricos permaneça uma tarefa eficiente.

Capítulo 3

Levantamento e Análise de Trabalhos e Ferramentas Relacionados

O presente capítulo tem como objetivo analisar trabalhos e softwares que se relacionam diretamente com a proposta apresentada neste trabalho. Na Seção 3.1 são discutidos o trabalho de Aldrich (2015) sobre a modelagem de redes de pré-requisitos curriculares. Em seguida a Seção 3.2 e a Seção 3.3 exploram softwares que oferecem perspectivas sobre grafos na visualização de currículos e a Seção 3.4 examina técnicas de visualização para análise de históricos escolares.

3.1 Rede de Pré-Requisitos Curriculares

Como mencionado na Seção 2.2, o trabalho de Aldrich (2015) apresenta uma metodologia abrangente para a construção e análise de redes de pré-requisitos curriculares, utilizando como estudo de caso os catálogos de cursos de graduação da Benedictine University (Illinois, EUA) entre 2009 e 2010. A metodologia se baseia na aplicação de métricas desenvolvidas na teoria dos grafos, com o objetivo de revelar um processo para avaliar a arquitetura curricular de forma holística, usando tecnologias modernas.

Neste trabalho, a extração de dados foi feita por *web scraping* do site da universidade usando Python e foram convertidas para o formato de rede usando NetworkX (versão 1.8), assim modelados para a CPN. A visualização e análise foram feitas usando Pajek, Gephi e yEd, softwares utilizados para a criação, visualização e análise de redes de grafos.

A construção da CPN envolveu regras específicas para codificar os relacionamentos do catálogo em DAGs: As ligações de pré-requisito obrigatórias recebem o mesmo peso de 1.0 em cada, ou seja, se A e B fossem obrigatórios para C , os arcos $A \rightarrow C$ e $B \rightarrow C$ valem 1.0 cada. As ligações de pré-requisitos opcionais recebem o peso 1.0 distribuído igualmente, então, $A \rightarrow C$ tem 0.5 e $B \rightarrow C$ tem 0.5. Também é feito tratamento de correquisitos *hard*, que pode ser representado por um arco bidirecional ($A \leftrightarrow B$), mas que introduz um ciclo de 2 membros fazendo com que a CPN deixe de ser um DAGs. Para preservar a ideia da estrutura da CPN, Aldrich (2015) optou por interpretar todos os correquisitos como ligações de correquisito suaves ("*soft corequisite bindings*"), codificando-os como pré-requisito.

Nos catálogos da Universidade, os pares de correquisitos geralmente continham um curso de "teoria/palestra"(A) e um curso de "laboratório/aplicado"(B), assim o curso de teoria foi considerado conceitual ou temporalmente prioritário, e o correquisito foi recodificado como um pré-requisito: *Teoria(A) → Laboratorio(B)*.

Os resultados foram divididos entre topologia global (CPN da grade curricular de disciplinas de graduação da Benedictine University entre 2009 e 2010) e a topologia a nível do curso de Bioquímica e Biologia Molecular – BMB.

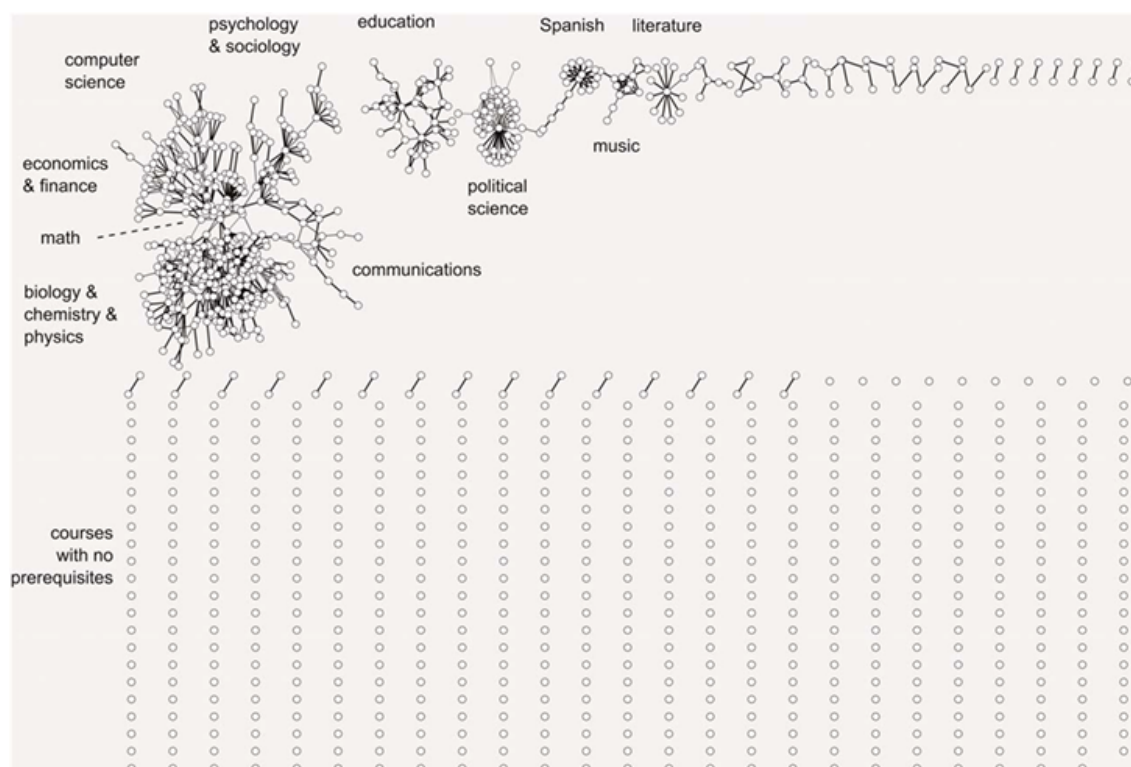


Figura 3.1: Disciplinas de graduação da Benedictine University de 2009-10 como um *Curriculum Prerequisite Network* (Renderizado por Pajek) . Fonte: (Aldrich, 2015)

A topologia geral (Figura 3.1) contém 1.097 disciplinas (os nós) e 770 ligações (arcos). O resultado da CPN se revelou altamente heterogêneo, devido à baixa conectividade média entre os nós (grau médio ponderado $k = 1.10$) e o fato de que mais da metade das disciplinas (51,0%) são isoladas.

A disciplina de *General Biology* (Biologia Geral) se mostrou numa posição influente com um grau de conectividade alto (disciplina "hub"), sendo pré-requisito direto de muitas outras disciplinas ($k = 29.0$). Todas as 10 principais disciplinas com maior centralidade de intermediação eram disciplinas de Ciência, Tecnologia, Engenharia e Matemática (STEM), e a que tem o valor mais alto de intermediação é *Computer Programming* (Programação de Computador), atuando como uma ponte crucial nos currículos.

A topologia a nível do curso BMB possui 29 disciplinas e 46 arcos, o grafo dela (Figura 3.2) difere as áreas das disciplinas em cores: Vermelho, matemática; laranja, física; amarelo, química; verde claro, ciências naturais; verde médio, bioquímica; verde escuro, biologia. Os arcos denotam relações de pré-requisito. A espessura dos arcos indica se o pré-requisito deve ser cursado (linha grossa) ou se outras disciplinas podem substituí-la como pré-requisito alternativo (linha fina; bordas dos nós (grossas vs. finas) representam as duas comunidades que foram identificadas usando o método Louvain no Gephi.

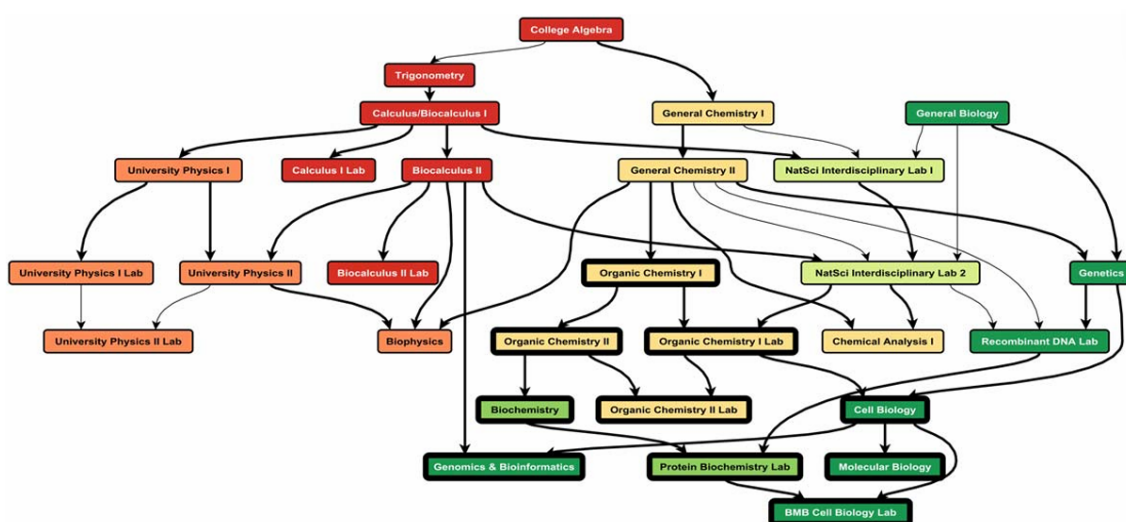


Figura 3.2: Grafo do curso de Bioquímica e Biologia Molecular da Benedictine University (Renderizado usando yEd). Fonte: (Aldrich, 2015)

As disciplinas de *General Chemistry II* (Química Geral II) e *Biocalculus II* (Cálculo aplicado à Biologia II) compartilham o grau ponderado mais alto ($k = 6.0$), ambos atuando como fontes de informação chave para o currículo, cada uma possuindo um único

pré-requisito e sendo chamada como pré-requisito direto por cinco outras disciplinas equivalentes (valores ponderados que permitem pré-requisitos alternativos). A disciplina com maior centralidade de intermediação é *Natural Science Interdisciplinary Lab II* (Laboratório Interdisciplinar de Ciências Naturais II), que canaliza uma quantidade substancial de informação entre diferentes segmentos do currículo.

Na Figura 3.3 os números, tamanhos dos nós e intensidades de cor dos nós indicam a centralidade de intermediação (variando entre 0 e 1,0) para cada disciplina. O layout é o mesmo da Figura 3.2.

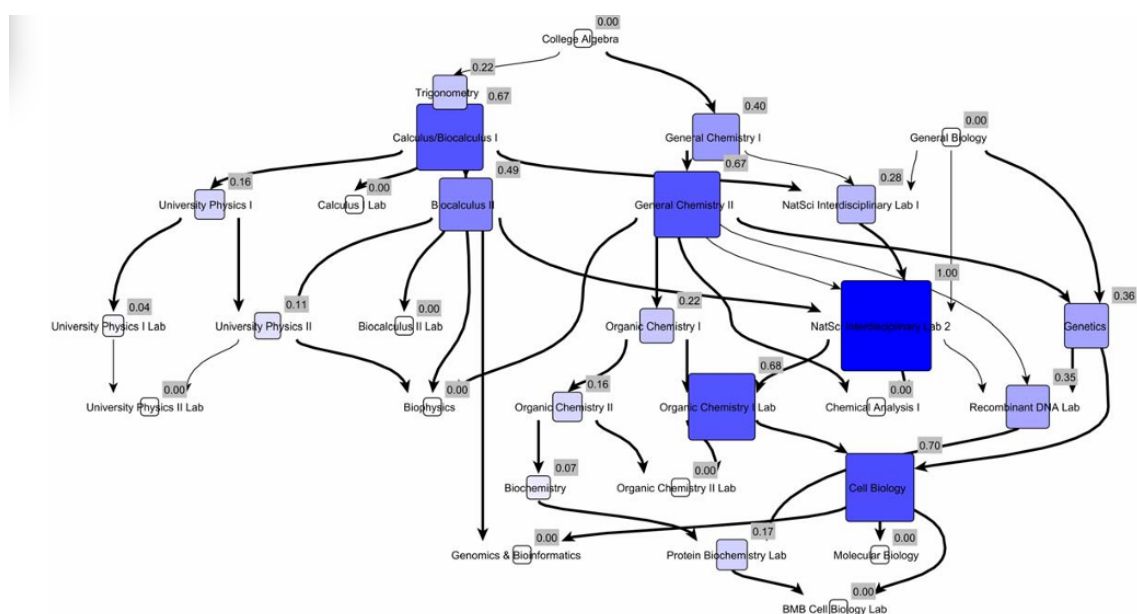


Figura 3.3: Centralidade das disciplinas do curso Bioquímica e Biologia Molecular (Renderizado usando o yEd). Fonte: (Aldrich, 2015)

Com todas as análises proporcionadas pela CPN, Aldrich (2015) entende que isso pode orientar reformas curriculares através de dois conceitos principais: integração curricular e coerência curricular. A integração curricular refere-se à conectividade adequada entre disciplinas, evitando o modelo fragmentado em que cada disciplina oferece sua própria informação isolada, buscando produzir currículos mais integrados e coerentes. A coerência curricular, por outro lado, diz respeito ao sentido lógico do currículo, subdividindo-se em coerência de fluxo progressivo (quando as disciplinas usam efetivamente a informação estabelecida pelo pré-requisito) e coerência de fluxo lateral (quando as disciplinas do mesmo semestre reforçam-se mutuamente em significado, ajudando as comunidades de aprendizagem dos estudantes)

Este trabalho representa uma contribuição fundamental para a área de análise curricular baseada em grafos. Sua abordagem demonstra como as metodologias e as métricas podem ser aplicadas para transformar a maneira como instituições de ensino interagem com a estrutura de seus próprios currículos e a compreendem.

3.2 Software Curricular Analytics

O Curricular Analytics é uma ferramenta de código aberto projetada para auxiliar instituições de ensino superior a visualizar e analisar a complexidade de suas estruturas curriculares (UC..., 2025). Desenvolvido em uma parceria entre a Damour Systems e Gregory L. Heileman, vice-reitor de assuntos acadêmicos e professor de engenharia elétrica e de computação da Universidade do Arizona, EUA, o software parte da premissa de que a complexidade estrutural de um currículo tem uma correlação direta e mensurável com o desempenho e o tempo de graduação dos estudantes. Conforme explicado por Heileman, a ferramenta foi criada para responder a perguntas críticas, como "quão complexo é um currículo?" e "como essa complexidade afeta os alunos?", transformando dados curriculares em insights acionáveis para a diretoria e a reformulação curricular (FUTURE..., 2021).

A adoção da ferramenta por diversas universidades, como a Universidade da Califórnia em San Diego – UCSD, a Universidade do Sul da Flórida e a Universidade do Estado do Colorado demonstra seu valor prático (CURRICULAR..., 2025).

Uma versão demo do software é disponibilizada de forma pública no site da UCSD. Na Figura 3.4 é mostrado o leiaute e o funcionamento dentro de um currículo do curso de bioengenharia.

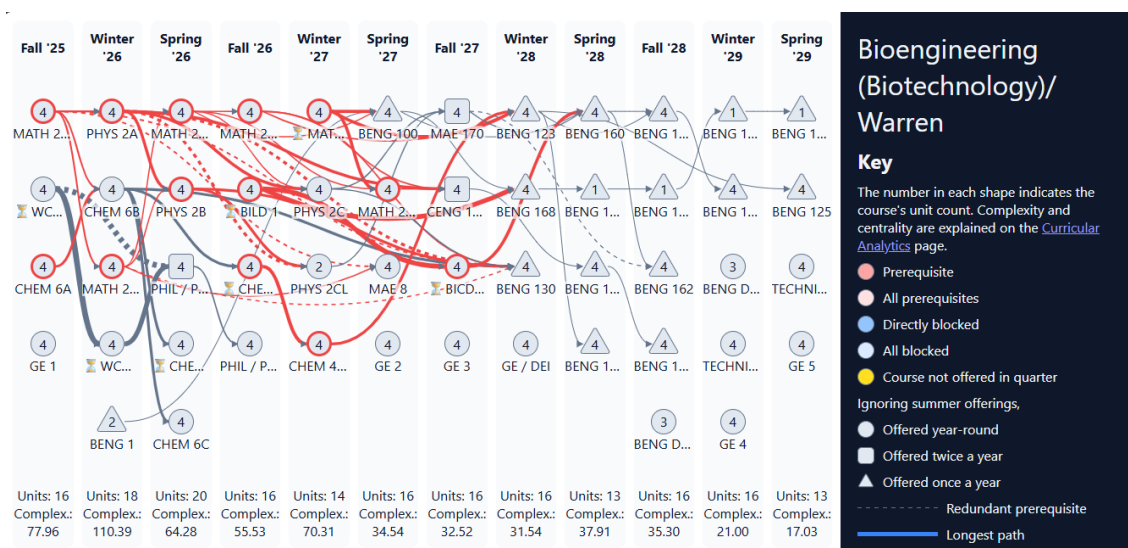


Figura 3.4: Demo do Curricular Analytics. Fonte: (UC..., 2025)

Como explicado no site oficial da ferramenta (CURRICULAR..., 2025), ela possui um conjunto de métricas quantitativas em cada nó (disciplina), como evidenciado na Figura 3.5, o que revela fortes convergências com os conceitos propostos por Aldrich (2015), junto com a teoria de grafos, como:

- **Fator de bloqueio (Blocking Factor):** Indica o número de disciplinas que são "bloqueadas", ou seja, dependem da aprovação da disciplina selecionada. Um fator de bloqueio alto significa que uma reprovação nesta disciplina impede o aluno de avançar em várias frentes do curso;
- **Fator de Atraso (Delay Factor):** Mede o comprimento do caminho mais longo de disciplinas que devem ser cursadas sequencialmente após a disciplina selecionada.
- **Complexidade (Complexity):** É a soma do fator de bloqueio e do fator de atraso, oferecendo um índice geral da criticidade da disciplina dentro do currículo.
- **Centralidade (Centrality):** A soma dos comprimentos de todos os caminhos longos que passam pela disciplina. Uma disciplina com alta centralidade atua como um "hub" e uma reprovação nela pode impactar profundamente a trajetória do estudante.
- **Taxa de DFW (DFW Rate):** Corresponde à porcentagem de estudantes que desistiram (Drop), reprovaram por nota (Fail) ou reprovaram por falta (Withdraw) na disciplina.

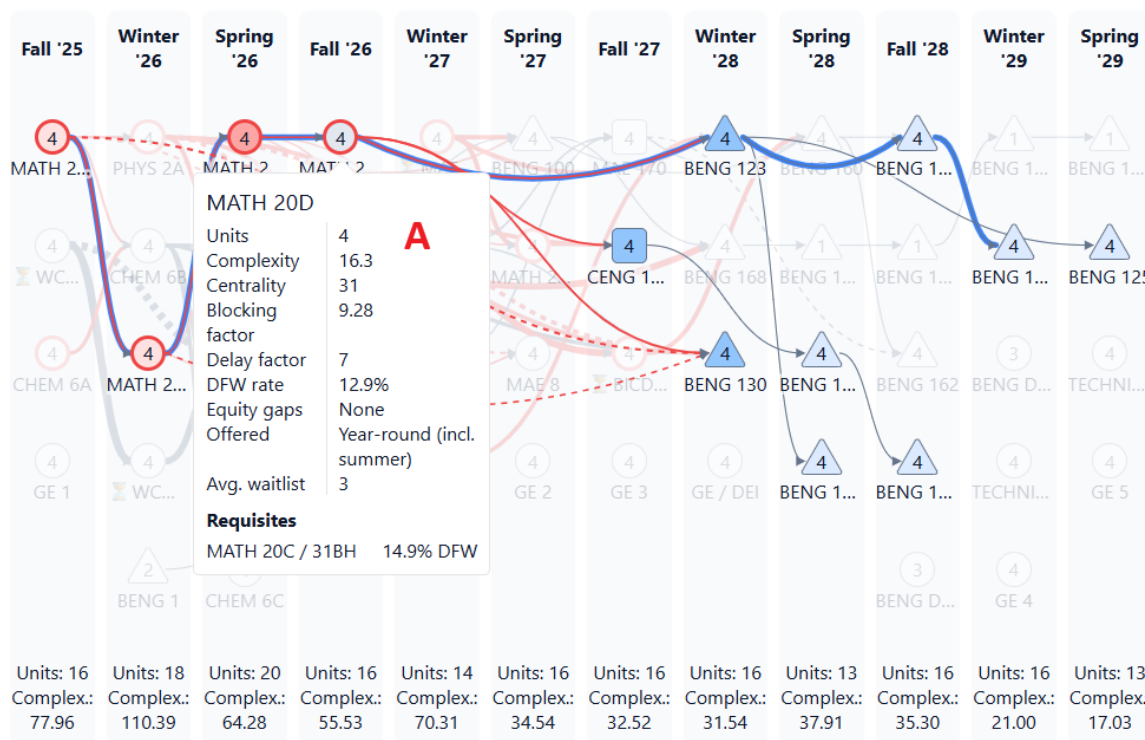


Figura 3.5: Em destaque: (A) Caixa informativa com as métricas da disciplina MATH 20D.

Fonte: (UC..., 2025)

O sistema é implementado na linguagem de programação Julia e utiliza Jupyter Notebooks para suas análises. As disciplinas, os catálogos e os requisitos são inseridos manualmente pela universidade ou importados por arquivo CSV no template padrão do sistema (Free, 2024).

3.3 CourseViewer

Uma ferramenta de particular interesse para este trabalho é o CourseViewer (Silva; Inoue; Mendonça, 2012), um software desenvolvido no Laboratório *Software Engineering and Information Systems* (SEIS) da Faculdade de Tecnologia/Universidade Estadual de Campinas, com o objetivo de auxiliar alunos, professores e coordenadores de curso no entendimento da organização e interdependência de disciplinas dos cursos de graduação.

A ferramenta é programada em Java e utiliza arquivos no formato XML para carregar os catálogos de cursos. O núcleo de sua interface é uma representação visual da matriz curricular baseada em um diagrama vértice-aresta, que corresponde a um grafo acíclico direcionado em camadas (LDAG).

Seguindo os princípios de Visualização de Informação, a ferramenta realiza um mapeamento visual dos dados acadêmicos para estruturas gráficas. Nessa estrutura, também são utilizados os vértices (nós), que são as disciplinas; as arestas, representando as relações de pré-requisito; e as camadas, onde as disciplinas são organizadas verticalmente criando uma linha do tempo. O posicionamento horizontal dos nós em cada camada é otimizado para reduzir o número de cruzamentos de arestas, um desafio clássico em visualização de grafos, utilizando técnicas como *Edge bundling*, para agrupar arestas em feixes e diminuir a poluição visual.

O CourseViewer oferece um conjunto de funcionalidades interativas para exploração e planejamento curricular, como a "visualização por impacto", na qual as disciplinas são coloridas com tons de vermelho e verde para destacar o número de disciplinas que a têm como pré-requisito; a análise de histórico escolar individual, que utiliza um código de cores para representar o desempenho em cada disciplina; e o planejamento interativo, no qual o usuário pode manipular a matriz curricular, adicionando ou removendo semestres e arrastando disciplinas entre eles.

A Figura 3.6 representa a interface do CourseViewer, que contém: um grafo em camadas (semestres) com as disciplinas oferecidas em um dado curso e seus pré-requisitos; uma seção que numera os semestres em cada linha, mostrando os créditos e o coeficiente de progressão adquirido naquele semestre; uma seção que indica opções de catálogos e cursos a serem vistos, bem como disciplinas a serem adicionadas no grafo; e, por fim, dois botões na parte inferior que possibilitam a adição e a remoção de semestres no grafo.

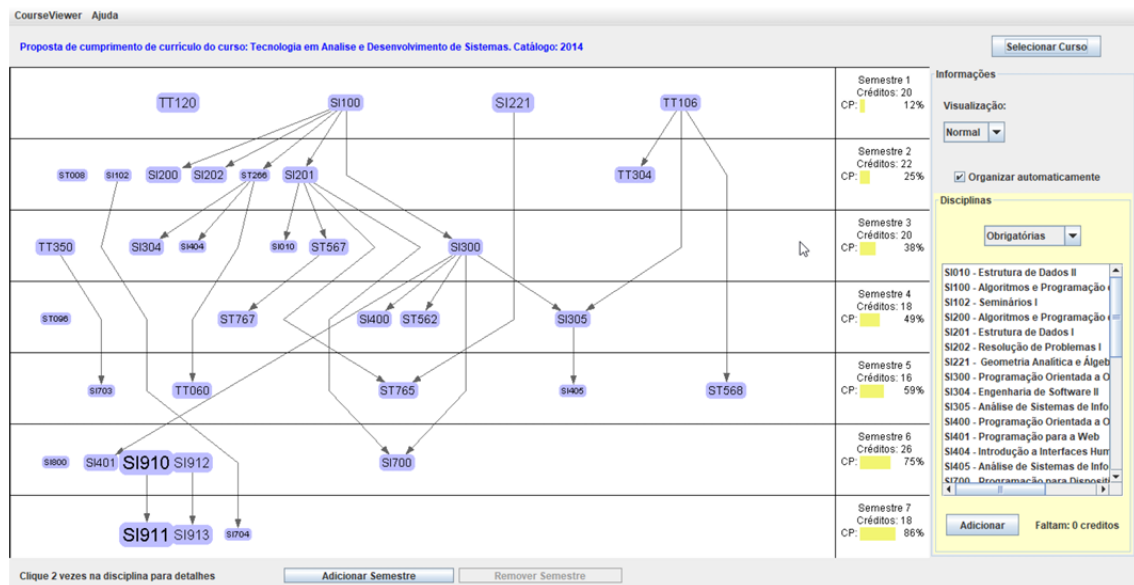


Figura 3.6: Software CourseViewer. Fonte: (Praça, 2019)

O trabalho de conclusão de curso de Praça (2019) conduziu um teste de usabilidade do CourseViewer, revelando pontos fortes e fracos sob a perspectiva da Interação Humano-Computador. Os testes concluíram que o software possui um bom nível de usabilidade e atende aos seus objetivos principais, porém a avaliação heurística apontou algumas violações em princípios de design de interface a serem resolvidos em versões futuras:

- Violação da Heurística "Reconhecimento ao invés de relembança";
- Violação da Heurística "Compatibilidade com o Mundo Real";
- Problemas de Design Visual.

O trabalho de Palomo (2019) também explorou o CourseViewer e focou na resolução de problemas estéticos e legibilidade dos grafos, dando continuidade a trabalhos anteriores. Alguns pontos explorados foram a legibilidade dos grafos, pois, apesar de técnicas como a heurística baricêntrica e o *Layer-by-Layer Sweep* terem sido implementadas para minimizar os cruzamentos de arestas, as arestas longas ainda apresentavam quebras em segmentos menores por meio de *dummy nodes* (nós virtuais). No entanto esses nós auxiliares causavam "dobras" ou mudanças abruptas de direção (zigue-zague) nas arestas, o que podia dificultar o entendimento do grafo por parte do usuário por conta da desorganização visual do grafo. Para isso, Palomo (2019) propôs o uso de uma técnica de InfoVis chamada *Edge Bundling* para

agrupar arestas e melhorar a estética do grafo de disciplinas, reduzindo a desorganização visual.

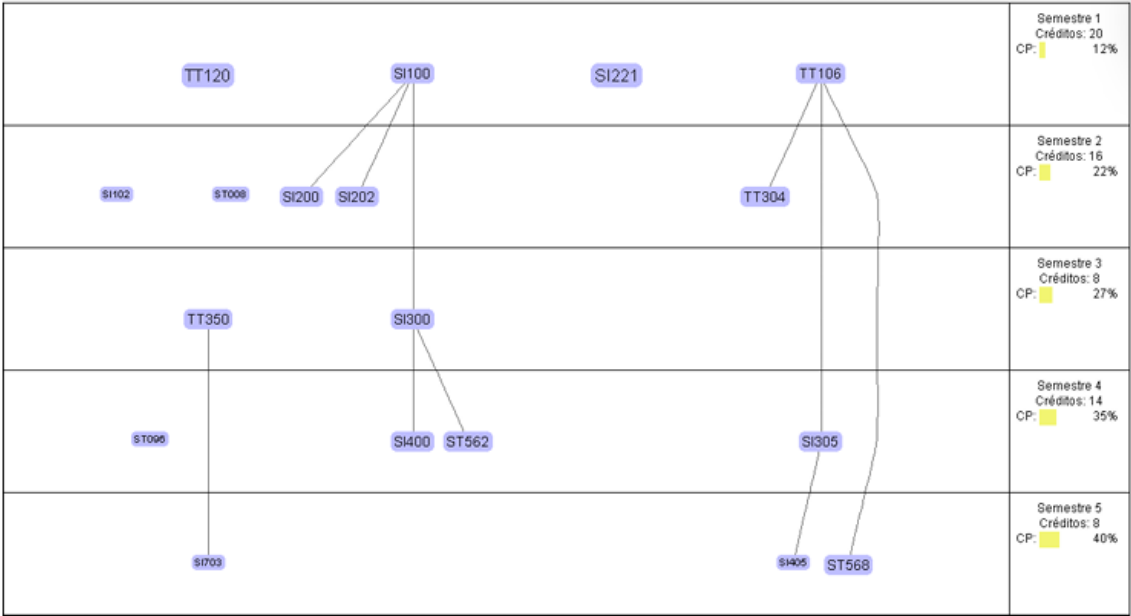


Figura 3.7: Currículo no CourseViewer sem *edge bundling* e com curvas cardinais. Fonte: (Palomo, 2019)

A Figura 3.7 mostra um exemplo de um grafo que não usa o *edge bundling* em suas arestas.

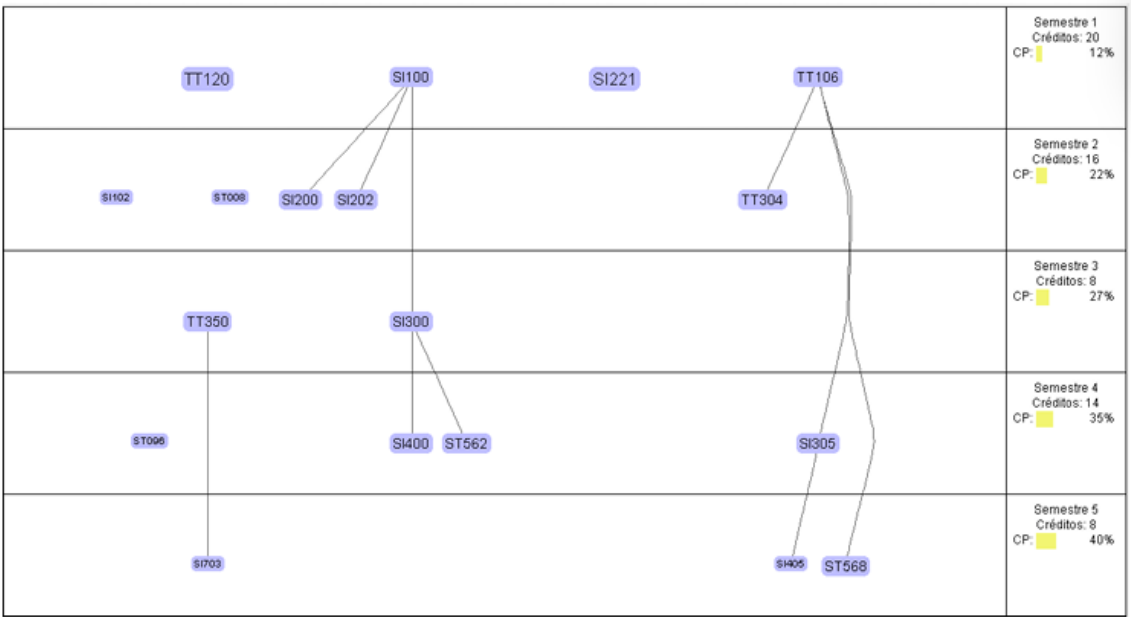


Figura 3.8: Currículo no CourseViewer com *edge bundling* e com curvas cardinais. Fonte: (Palomo, 2019)

Além da implementação do *Edge Bundling* (Figura 3.8), as Curvas Cardinais foram escolhidas para suavizar o desenho das arestas longas, que ainda eram desenhadas como linhas poligonais abertas. Estas permitem que a curva passe suavemente por pontos de controle (interpolação).

3.4 Uso de Matrizes Reordenáveis e Mapas de Calor para Análise de Históricos Escolares

O trabalho de Barros, Mendes e Silva (2019) propõe uma abordagem para análise visual de históricos escolares de estudantes, utilizando matrizes reordenáveis e mapas de calor (*heatmaps*). Ele parte do pressuposto que as análises tradicionais de históricos dificultam a identificação de padrões e situações atípicas - como estudantes com desempenhos inconsistentes ou disciplinas com baixo rendimento geral. Por isso é importante o uso de ferramentas que facilitam o trabalho analítico dos coordenadores de curso dentro do contexto de EDM.

O mapa de calor é uma forma de visualização gráfica que pode representar grandes quantidades de dados em um espaço reduzido. Barros, Mendes e Silva (2019) utilizam o mapa de calor com uma estrutura tabular onde cada linha representa um estudante, cada coluna representa uma disciplina e a cor de cada célula representa a nota obtida pelo estudante naquela disciplina correspondente, que varia do vermelho escuro (nota 0) ao azul escuro (nota 10) (Figura 3.9).

Uma matriz reordenável é a estrutura de dados subjacente ao mapa de calor. A reordenação pode ser manual ou automática e uma boa permutação pode revelar similaridades nos dados que estariam ocultas.

O sistema implementou o algoritmo para reordenamento *Optimal Leaf Order* (OLO), da biblioteca Reorder.js. Ele inicia com um agrupamento hierárquico das linhas (ou colunas) da matriz e encontra uma ordem que seja consistente com o dendrograma (a estrutura da árvore) gerado por esse agrupamento e que minimiza a soma das distâncias entre linhas (ou colunas) consecutivas. Para o rearranjo no mapa de calor acontecer, é preciso fazer outro tratamento dos dados com a biblioteca D3.js, que gera duas matrizes de distância (uma para as linhas e outra para as colunas), as quais são fornecidas ao método OLO como parâmetros. A escolha

deveu-se ao fato de ele proporcionar bons resultados utilizando apenas dois parâmetros para a realização do agrupamento: a métrica de distância e o tipo de ligação (*linkage type*).

Ao aplicar o OLO, o sistema pode agrupar alunos distintos com base em seu desempenho geral (por exemplo, estudantes com as melhores notas na parte superior e piores notas na parte inferior). Ele também permite ordenar as linhas ou colunas de acordo com as notas de uma disciplina ou de um aluno específico.

As funcionalidades e a visualização de dados da ferramenta foram desenvolvidas seguindo o Mantra de Busca de Informações Visuais de Shneiderman (1996), que pode ser resumido para "Visão geral primeiro, zoom e filtro, depois detalhes sob demanda" ("*overview first, zoom and filter, then details-on-demand*"). O sistema possui uma visão geral e manipulação de zoom e detalhes sob demanda ao passar o mouse sobre uma célula representando a nota.

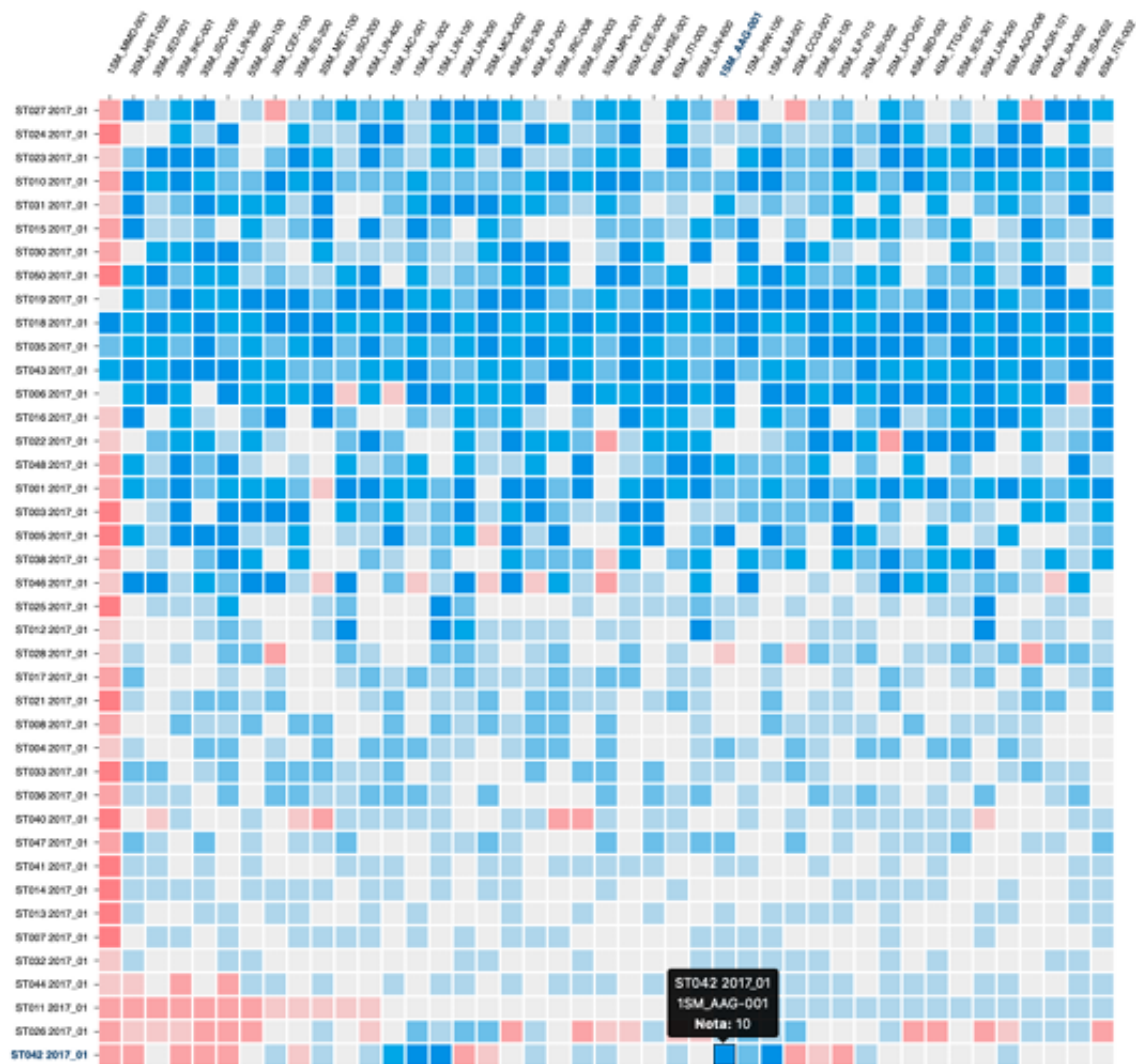


Figura 3.9: Gráfico ordenado pelas notas do aluno ST011 (terceira linha na direção de baixo para cima). Na parte inferior, uma janela de detalhes com a nota do aluno ST042 na disciplina 1SM AAG 001. Fonte: (Barros; Mendes; Silva, 2019)

O sistema lê as notas e os identificadores de alunos e disciplinas em um formato de arquivo CSV. Ele usa a biblioteca D3.js que permite exibir dados na forma de gráficos usando HTML (para conteúdo da página), SVG (para gráficos vetoriais), CSS (para estética) e JavaScript (para interação).

Com isso, os autores demonstram, por meio de um conjunto de dados sintético composto por 40 estudantes, diversos padrões que podem ser identificados através da visualização em mapa de calor. Por exemplo, uma visualização de coluna predominantemente vermelha indica uma disciplina na qual a maioria dos estudantes obteve notas medianas e baixas — ou seja, sob a perspectiva dos alunos, essa é uma disciplina difícil no curso. Uma visualização de

um conjunto de linhas azuis indica alunos com boas notas em quase todas as disciplinas, um padrão que pode ajudar coordenadores e professores a selecionar alunos com esse desempenho para atividades acadêmicas. Outro padrão que pode ser identificado, quando as disciplinas estiverem ordenadas temporalmente, é a ocorrência de desempenhos ruins em um período específico da trajetória acadêmica de um estudante (por exemplo, notas baixas no terceiro semestre). Essas linhas vermelhas podem alertar sobre possíveis momentos de dificuldades pessoais e/ou psicológicas.

Desse modo, o trabalho de Barros, Mendes e Silva (2019) destaca como os mapas de calor são eficazes para identificar padrões de desempenho e comparações diretas de notas entre estudantes e disciplinas, enquanto os LDAGs permitem uma compreensão mais profunda da estrutura curricular e das relações entre as disciplinas. Uma integração de ambas as técnicas poderia proporcionar uma ferramenta mais completa e versátil para análise de históricos escolares.

Capítulo 4

Metodologia

Este trabalho foi conduzido junto com o grupo de pesquisa de EDM coordenado pelo Prof. Dr. Celmar Guimarães da Silva, na Faculdade de Tecnologia da UNICAMP tomando como base um desenvolvimento exploratório e incremental, com foco na construção de um protótipo funcional utilizando técnicas de visualizações baseadas em LDAGs.

Na Seção 4.1 foi feito o levantamento de requisitos do sistema, levando em conta as tarefas do usuário. A Seção 4.2 aborda o sistema GradeGen e suas configurações para gerar simulações de históricos escolares. A Seção 4.3 define as técnicas de visualização usadas na proposta deste trabalho. A Seção 4.4, a Seção 4.5 e a Seção 4.6 explicam as bibliotecas usadas no código, o desenvolvimento do código como um todo e a visualização proposta. Por fim, a Capítulo 5 apresenta os resultados observados na visualização.

A Figura 4.1 apresenta uma visão geral do fluxo de desenvolvimento do sistema proposto, destacando as etapas conduzidas neste trabalho e a utilização de módulos externos como fontes de dados.

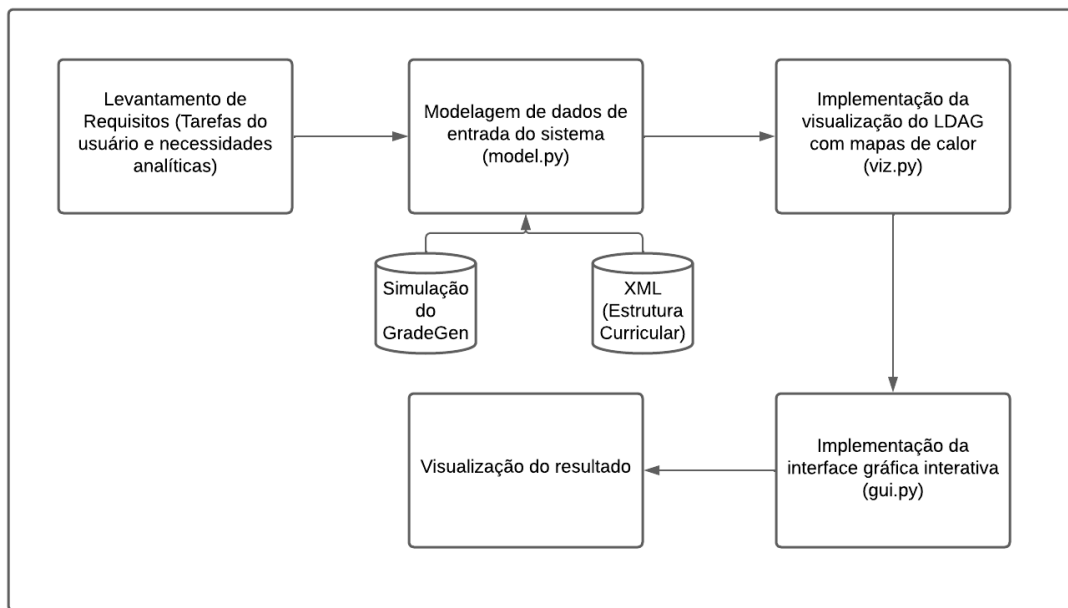


Figura 4.1: Fluxo de desenvolvimento do software proposto, destacando as etapas de levantamento de requisitos, modelagem de dados, implementação da visualização, interface gráfica e utilização de fontes externas de dados. Fonte: Autora.

4.1 Levantamento de Requisitos

Como visto no Capítulo 3, foi realizado um levantamento e análise de trabalhos e ferramentas que se relacionam com a proposta deste trabalho.

Nele foi possível mostrar a relevância de visualizações de dados mais analíticas sobre os currículos universitários.

Através da análise da topologia da rede e dos históricos escolares é possível identificar quais disciplinas têm um impacto significativo no tempo de graduação e podem atuar como um gargalo. E os mapas de calor com os históricos escolares ajudam na fácil visibilidade de padrões de desempenho, identificação de disciplinas mais difíceis e a detecção de correlações entre os estudantes e as disciplinas.

Como parte do trabalho de mestrado de Thiago Gonçalves Mendes no grupo de pesquisa, foram levantadas uma série de tarefas para os quais os professores universitários gostariam de ter suporte. A Tabela 4.1 sintetiza as principais tarefas identificadas:

Tabela 4.1: Tarefas Relevantes para o Usuário

Tarefa	Objetivo
Visualizar	Descoberta de padrões e situações, como, pendências de alunos no cumprimento de disciplinas, ver padrões de desempenho de alunos e ver caminhos críticos de dependências entre disciplinas
Comparar	Análise relativa entre elementos como disciplinas, notas e históricos dos alunos
Identificar	Deteção de casos específicos, como alunos com mudanças abruptas de desempenho
Verificar	Confirmar situações desconhecidas, como necessidade de vagas em disciplinas para próximos semestres
Analisar	Acompanhamento de métricas, como, o desempenho dos alunos por semestre

Além das tarefas funcionais, foram identificados requisitos não-funcionais importantes para a ferramenta:

- Usabilidade: A interface deve ser fácil de ser usada, seguindo princípios de Interface Humano-Computador – IHC;
- Desempenho: Capacidade de processar históricos escolares de turmas de pelo menos 30 alunos;
- Suporte à visualização de currículos de diferentes complexidades e quantidades de disciplinas.

Estes requisitos orientaram o desenvolvimento da solução proposta, garantindo que as visualizações implementadas atendam às necessidades dos usuários e preencham um "gap" no quesito de análise curricular.

4.2 Sistema GradeGen

O GradeGen é uma ferramenta criada para gerar conjuntos de dados sintéticos de históricos escolares para uso em softwares de mineração de dados educacionais desenvolvido na FT/UNICAMP utilizando Python, JavaScript, HTML e Firebase (Oliveira, 2020).

O módulo central que gera os dados tabulares resultantes da simulação do comportamento dos alunos fictícios é o motor de simulação, que processa as entradas do usuário recebidas através de uma interface web baseada no microframework Flask.

Ele opera sobre os dados usando o conceito de vetores e matrizes, e utiliza a biblioteca pandas para a manipulação. Primeiro, é identificada a quantidade de alunos a serem simulados (que ocupam o índice da matriz) e a quantidade de disciplinas (podendo ser importadas de um arquivo XML contendo o currículo completo de um curso). Para cada disciplina são atribuídas três colunas: nota obtida pelo aluno, frequência do aluno na disciplina e a turma em que foi cursada.

O motor de simulação avança na seleção das próximas disciplinas que o aluno deve cursar, respeitando o semestre de oferta e os pré-requisitos necessários. Utilizando um algoritmo com estratégia gulosa, ele define a quantidade de disciplinas atribuídas, garantindo que cada aluno curse o máximo de créditos possíveis no semestre. O algoritmo verifica: se o aluno já obteve aprovação na disciplina; se o aluno obteve aprovação nos pré-requisitos e se a quantidade de créditos da disciplina excede o limite máximo de créditos permitido para o aluno no semestre.

As notas e frequências geradas na simulação são influenciadas por múltiplos fatores e configurações definidas pelo usuário. A função principal de geração de simulação aceita diversos parâmetros importantes, como o de definição de grupos de alunos (Tabela 4.2):

Tabela 4.2: Parâmetros de grupos de alunos que regem o desempenho. Fonte: (Oliveira, 2020)

Parâmetro	Descrição
<i>parameter_name</i>	Identificador único do grupo (ex.: "Acima da média")
<i>min_grade</i>	Nota mínima a ser sorteada para o grupo (de 0.00 a 10.00)
<i>max_grade</i>	Nota máxima a ser sorteada para o grupo (de 0.00 a 10.00)
<i>qtde</i>	Quantidade de alunos que pertencerão a este grupo

Também existem os fatores de alteração, mecanismos aleatórios que modificam as notas e frequências sorteadas, simulando eventos do mundo real (Tabela 4.3):

Tabela 4.3: Fatores de alteração (impacto). Fonte: (Oliveira, 2020)

Fator	Descrição
Sabotagem/Recuperação de Notas (<i>gradeSabRecFactors</i>)	Períodos aleatórios nos quais notas de alunos sorteados sofrem impacto positivo ou negativo. O usuário pode configurar o valor de impacto negativo e positivo, e a porcentagem de alunos afetados
Sabotagem/Recuperação de Frequência (<i>frequencySabRecFactors</i>)	Períodos aleatórios nos quais frequências de alunos sorteados sofrem impacto positivo ou negativo. O usuário pode configurar o valor de impacto negativo e positivo, e a porcentagem de alunos afetados
Disciplinas Fáceis/Díficeis (<i>factors, easyPasses, hardPasses</i>)	Disciplinas definidas como fáceis recebem acréscimo na nota da turma, e difíceis recebem decréscimo. O usuário configura o fator de impacto positivo e o fator de impacto negativo.
Alteração Abrupta na Nota da Turma (<i>factorsEasyHard</i>)	Casos aleatórios e pontuais em que toda uma disciplina tem um acréscimo ou decréscimo considerável na nota final. O usuário pode configurar os valores máximos (em módulo) para impacto positivo ou negativo (queda e aumento brusco)

A estrutura de saída do GradeGen fornece um arquivo em CSV contendo os históricos completos dos alunos simulados. Um exemplo de parte desse arquivo pode ser visto na Figura 4.2.

	Turma	EB101	Freq	Turma	SI100	Freq	Turma	SI101	Freq	Turma	SI102	Freq	Turma	ST008	Freq	Turma	TT106	Freq	Turma	SI120	Freq
162911	A	4.94	77.77	A	0.00	64.07	A	2.42	74.27	B	0.84	70.62	B	0.00	64.00	A	2.14	85.16	A	4.48	82.97
165092	A	3.62	86.56	A	1.75	89.48	B	0.25	83.95	B	4.55	87.97	B	3.06	92.92	B	3.48	96.31	A	1.95	71.26
111062	A	9.61	65.45	A	8.80	70.94	A	0.00	63.31	B	8.92	93.38	A	8.28	85.72	A	8.71	80.42	A	7.35	75.47
149674	A	5.36	68.98	A	6.07	97.24	A	6.71	79.74	B	5.66	66.00	B	0.00	64.47	A	5.60	98.52	A	5.07	93.67
181655	A	1.53	79.50	B	2.71	98.69	A	0.00	61.89	A	4.85	93.23	B	2.63	98.06	A	2.32	70.27	A	0.81	84.69
184551	A	1.95	87.38	B	2.03	72.07	B	2.48	87.66	B	0.10	82.11	B	4.74	85.65	B	0.00	62.00	A	2.48	70.24
122699	A	0.00	62.70	A	0.84	73.11	A	4.70	82.80	A	3.45	65.37	A	3.50	97.74	A	0.61	70.58	A	4.27	85.18
173139	A	2.96	77.89	B	2.30	97.97	A	3.33	96.08	A	4.02	73.25	B	3.38	87.12	A	1.70	76.85	A	1.14	97.96
181126	A	1.08	68.76	A	3.27	77.73	B	3.37	84.24	B	0.88	82.02	A	0.16	85.43	B	3.33	70.61	A	1.53	91.64
123953	A	6.17	82.05	B	5.58	65.70	B	6.58	76.02	B	6.32	82.46	A	5.73	65.40	A	6.04	85.68	A	0.00	60.68
147329	A	7.43	80.61	A	9.14	98.56	B	8.73	66.88	B	9.05	68.11	B	8.12	92.36	B	7.99	77.31	A	8.92	71.73
123309	A	8.43	84.96	A	7.08	66.16	A	0.00	63.34	A	7.95	86.00	A	8.66	80.56	B	0.00	60.40	A	8.62	73.22
131562	A	5.76	91.24	B	5.53	90.47	B	5.98	71.47	B	6.35	97.98	B	5.84	84.64	A	6.16	77.12	A	6.94	77.48
105532	A	8.59	86.96	B	8.93	67.28	B	7.48	72.81	B	9.11	91.13	B	8.21	89.23	A	7.54	71.23	A	8.52	92.11
192345	A	3.91	74.44	B	4.24	81.92	B	0.51	81.68	A	4.12	99.39	A	1.76	75.87	B	2.97	66.50	A	2.76	83.37
165179	A	0.41	85.76	B	0.00	90.44	A	4.60	95.41	B	0.00	63.80	B	3.03	66.99	A	1.30	83.38	A	2.35	83.36
145795	A	0.00	63.86	B	2.27	88.77	B	2.95	89.63	A	3.42	84.84	B	4.06	88.78	A	2.64	80.89	A	0.00	61.42
196580	A	6.54	65.55	B	6.97	94.69	A	5.99	97.53	A	6.50	91.72	A	5.33	86.80	A	5.16	68.99	A	6.20	89.88
121437	A	6.48	94.66	B	7.00	70.68	B	0.00	63.87	A	5.46	80.16	B	5.06	83.36	B	6.18	69.36	A	5.12	92.04
153469	A	6.12	93.72	A	5.36	76.77	A	6.01	77.19	A	6.38	79.82	A	5.97	94.26	B	6.43	66.02	A	5.67	80.57
182119	A	2.18	99.80	B	2.84	82.75	A	0.50	88.91	B	4.97	88.94	A	0.00	62.05	B	2.05	83.49	A	4.25	88.76
131372	A	5.96	71.27	B	6.85	96.96	B	6.08	65.84	A	5.21	76.71	A	6.33	76.76	B	6.86	71.93	A	6.03	91.53

Figura 4.2: Exemplo de uma simulação gerada pelo GradeGen. A primeira coluna corresponde aos RAs dos estudantes; em seguida, as colunas Turma, Nota e Frequência se repetem para cada disciplina.

Com os parâmetros utilizados no sistema e a geração de conjuntos de dados com padrões conhecidos, as simulações do GradeGen facilitam a validação de testes de visualizações diversas, como gerar dados nos quais a turma possui um grupo de alunos com dificuldades em disciplinas, e posteriormente, verificar se a visualização proposta consegue destacar esse padrão de forma eficaz.

4.3 Definição das técnicas de visualização

Em conjunto com o grupo de pesquisa EDM e considerando o levantamento de trabalhos e os requisitos dos usuários, foram definidas as técnicas de visualização que melhor se adequariam ao problema proposto. A estrutura de um DAG é útil para observar hierarquias complexas que se relacionam diretamente com ligações de pré-requisitos entre disciplinas, e um LDAG adiciona o elemento temporal para a análise, pensando nos semestres letivos e oferecimentos de disciplinas em diferentes períodos do ano letivo. Por esses motivos, os LDAGs foram escolhidos como estrutura visual principal para visualização dos currículos.

Os mapas de calor foram escolhidos para exibição das notas dos estudantes em cada disciplina específica do grafo. Esta integração combina as vantagens dos LDAGs, que

preservam a estrutura curricular de pré-requisitos e progressão temporal, com as vantagens dos mapas de calor, que facilitam a visualização e comparação de desempenho acadêmico.

Com base no software Curricular Analytics (CURRICULAR..., 2025), também foi incorporada a visualização de grafo da esquerda para a direita, com as colunas dos semestres na vertical e as ligações entre os nós na horizontal, criando assim uma linha do tempo.

4.4 Bibliotecas Computacionais

O sistema foi desenvolvido utilizando a linguagem Python, escolhida por sua ampla adoção na comunidade de visualização de dados e pela disponibilidade de bibliotecas especializadas.

A biblioteca NetworkX foi usada para construir e gerenciar a representação em grafo do currículo acadêmico. Ela oferece estruturas de dados eficientes para grafos direcionados (DiGraph), permitindo a associação de atributos customizados tanto aos nós quanto às arestas. A biblioteca Matplotlib é responsável pela renderização visual do grafo; ela permite a criação de figuras (Figure) e eixos (Axes) customizáveis sobre os quais são desenhados os componentes do LDAG. Recursos como a *FancyBboxPatch* são utilizados para desenhar as caixas arredondadas que representam os nós das disciplinas, e a *FancyArrowPatch* permite criar as setas estilizadas das relações dos nós. A Matplotlib também oferece suporte nativo para interatividade com eventos de mouse, usados para cliques nos nós e destaque de elementos ao passar o cursor.

Para a estrutura de dados do sistema foi utilizada a biblioteca Pandas, que foi responsável pela leitura e processamento de dados dos arquivos CSV gerados pelo GradeGen. Essa biblioteca facilita operações de renomeação de colunas, tratamento de valores ausentes, filtragem de dados e identificação de registros únicos.

Algumas bibliotecas padrões de Python também foram utilizadas, como: *xml.etree.ElementTree*, usada para processamento do arquivo XML contendo a estrutura do currículo acadêmico com os códigos da disciplina, nome, créditos, semestres de oferta e relações de pré-requisito; a biblioteca *Tkinter*, para criação de interfaces gráficas de usuário (*Graphical User Interface* – GUI) como janela principal, barra lateral de informações e elementos de controle; a biblioteca *NumPy*, usada para operações matemáticas e vetoriais para cálculo de distâncias e determinar pontos de início e fim das setas; e a biblioteca

Collections (*defaultdict*) usada para agrupar disciplinas a um dicionário sem necessidade de verificar previamente se a chave (semestre) já existe, simplificando o código.

4.5 Arquitetura do Software

A arquitetura do sistema foi dividida em três módulos principais: a estrutura de dados (*model.py*), visualização que converte os dados da model em uma figura estática (*viz.py*) e a interface gráfica, que hospeda a figura, adiciona controles e gerencia eventos de interação (*gui.py*).

4.5.1 Módulo *model.py*

No modelo de dados (*model.py*) foram desenvolvidas duas classes principais: a classe *Disciplina* e a classe *Catalogo*. A classe *Disciplina* (Figura 4.3) contém o código da disciplina, o nome por extenso, os créditos, o semestre que é oferecida, a lista de pré-requisitos e a lista de dicionários dos históricos de alunos nela.

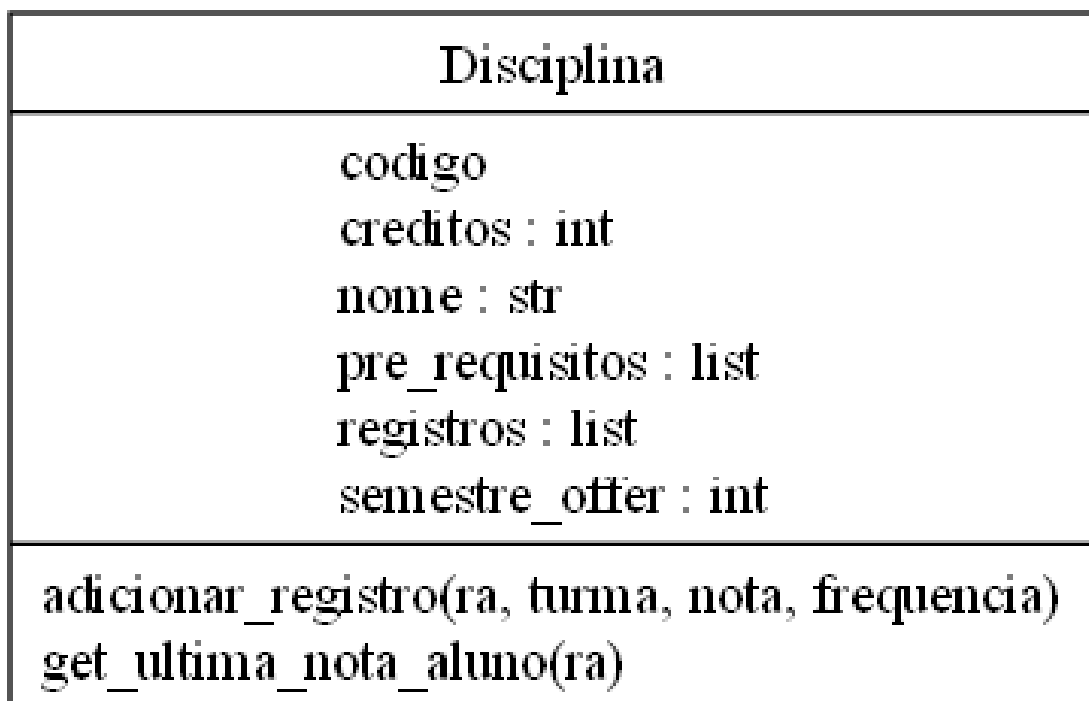


Figura 4.3: Diagrama da classe *Disciplina* no *model.py*. Fonte: Autora

Ela possui dois métodos importantes: *adicionar_registro* que para cada aluno registra o RA, a turma em que estavam, sua nota e sua frequência; e o *get_ultima_nota_aluno*, que retorna o

último registro do aluno, assim se o aluno cursou uma mesma disciplina 3 vezes a visualização mostrará a nota mais recente (*registros_aluno[-1]*, como mostrado na Figura 4.4) obtida por ele.

```
def adicionar_registro(self, ra, turma, nota, frequencia):
    self.registros.append({
        "ra": ra,
        "turma": turma,
        "nota": nota,
        "frequencia": frequencia
    })

def get_ultima_nota_aluno(self, ra):
    registros_aluno = [r for r in self.registros if str(r['ra']) == str(ra)]
    if not registros_aluno:
        return None
    return registros_aluno[-1]
```

Figura 4.4: Métodos da classe Disciplina .Fonte: Autora

A classe Catalogo gerencia o conjunto completo de disciplinas (*self.disciplinas*) em um dicionário e inicializa o contador do total de alunos. Ela possui três métodos de carregamento dos dados.

O método *_carregar_de_xml* percorre o arquivo XML usando a biblioteca *xml.etree.ElementTree*. Ele itera por cada tag <subject> obtendo o código, nome, créditos, semestre oferecido e pré-requisitos, cria uma nova instância da classe Disciplina e a armazena no dicionário de *self.disciplinas*, usando o código como chave (Diagrama da Classe na Figura 4.5).

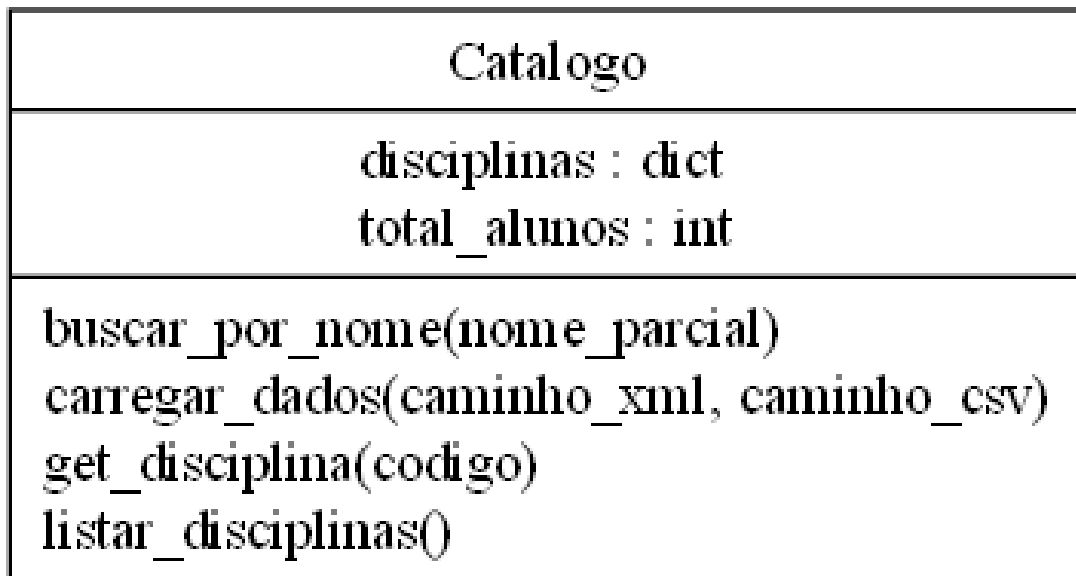


Figura 4.5: Diagrama da classe Catalogo. Fonte: Autora

O método `_carregar_de_csv` (Figura 4.6) abre e lê o arquivo CSV com a biblioteca `pandas`, renomeia a primeira coluna do arquivo para RA e calcula o total de alunos. Ele também itera em blocos de 3 colunas do `DataFrame`, para lidar com o padrão de colunas do CSV gerado pelo `GradeGen`. Para tratar as colunas duplicadas (a mesma disciplina aparece mais de uma vez) a biblioteca `pandas` adiciona sufixos, e então é realizada uma normalização de todos os sufixos para 0, juntando as duplicatas. Em seguida, ele itera por cada linha (aluno) desse bloco de colunas, adicionando o registro ao objeto da disciplina.

```

def _carregar_de_csv(self, caminho_csv):
    #Método privado para carregar os registros de alunos do CSV.
    try:
        df = pd.read_csv(caminho_csv)
        df.rename(columns={df.columns[0]: 'RA'}, inplace=True)
        self.total_alunos = df['RA'].nunique()
    except FileNotFoundError:
        print(f"Erro: O arquivo '{caminho_csv}' não foi encontrado.")
        return
    for i in range(1, len(df.columns), 3):
        if i + 2 >= len(df.columns): continue
        col_codigo_disciplina = df.columns[i+1]
        codigo_base = col_codigo_disciplina.split('.')[0]
        disciplina_obj = self.get_disciplina(codigo_base)
        if not disciplina_obj:
            continue
        #Itera sobre cada linha de aluno
        for _, row in df.iterrows():
            try:
                nota = float(row[col_codigo_disciplina])
                frequencia = float(row[df.columns[i+2]])
                disciplina_obj.adicionar_registro(
                    ra=row['RA'],
                    turma=row[df.columns[i]],
                    nota=nota,
                    frequencia=frequencia
                )
            except (ValueError, TypeError):
                continue

```

Figura 4.6: Método privado para percorrer o arquivo CSV. Fonte: Autora

Além disso há ainda 3 outros métodos nessa classe, o método *carregar_dados* é uma função wrapper (empacotadora) que chama os carregadores e imprime o status. O método utilitário de *listar_disciplinas* retorna a lista completa de todas as disciplinas (*self.disciplinas.values*). Por fim, o método de *get_disciplina* busca e retorna o código da disciplina.

Portanto, o modelo de dados utilizado no sistema armazena metadados das disciplinas do XML e os históricos dos alunos do CSV gerado pelo GradeGen.

4.5.2 Módulo viz.py

O módulo *viz.py* converte a estrutura de dados do Catalogo em uma visualização gráfica de grafo. Uma das principais funções desse módulo é a *criar_grafo_do_catalogo* que constrói um dígrafo usando a biblioteca de NetworkX (*nx.DiGraph*) e adiciona nós (nodes) e arestas (edges). A Figura 4.7 exemplifica como a NetworkX permite construir um grafo.

```

import networkx as nx
#Cria um dígrafo
curriculum_graph = nx.DiGraph()
#Adicionar os nós:
curriculum_graph.add_node("TCC001", nome="Programação I", credits=4, semestre=1)
curriculum_graph.add_node("MAT001", name="Calculo I", credits=6, semestre=1)
curriculum_graph.add_node("TCC002", name="Programação II", credits=4, semestre=2)
#Adicionar os pré-requisitos:
curriculum_graph.add_edge("TCC001", "TCC002") # TCC001 é pré-req de TCC002
curriculum_graph.add_edge("MAT001", "TCC002") # MAT001 é pré-req de TCC002

```

Figura 4.7: Exemplo de código para criação de um dígrafo no NetworkX. Fonte: Autora

Na função *criar_grafo_do_catalogo* do *viz.py* (Figura 4.8), o sistema percorre toda a lista de disciplinas do Catalogo e adiciona cada uma delas como nó no dígrafo G com os atributos de nome, semestre e créditos. Depois que todos os nós são adicionados, um segundo loop itera pela lista de novo e, para cada pré-requisito encontrado, ele adiciona uma aresta direcionada da disciplina de pré-requisito à disciplina atual.

```

def criar_grafo_do_catalogo(catalogo: Catalogo) -> nx.DiGraph:
    G = nx.DiGraph()

    # Adicionar os nós (GRAFO)
    for d in catalogo.listar_disciplinas():
        semestre = getattr(d, "semestre_offer", 0) or 0
        credits = getattr(d, "credits", 0) or 0
        nome = getattr(d, "nome", None)
        # chave do nó = código da disciplina
        G.add_node(d.codigo, nome=nome, semestre=semestre, credits=credits)
    # Adicionar os pre-requisitos
    for d in catalogo.listar_disciplinas():
        for pre in d.pre_requisitos:
            if G.has_node(pre):
                G.add_edge(pre, d.codigo)

    return G

```

Figura 4.8: Função *criar_grafo_do_catalogo*. Fonte: Autora

Após a construção do dígrafo, é preciso desenhá-lo em uma visualização gráfica (usando a *Figure* da biblioteca Matplotlib). Aqui se encontra a parte mais crucial do núcleo visual, a função *desenhar_grafo_em_camadas*, ela é dividida em várias etapas, a primeira sendo o agrupamento de nós por semestre, definir os parâmetros de leiaute e calcular as coordenadas dos nós.

```

def desenhar_grafo_em_camadas(grafo: nx.DiGraph, catalogo: Catalogo,
                              ordem_heatmap='decrecente', ra_sequencia=None) -> Figure:
    # Agrupar nós por semestre
    nodes_por_sem = defaultdict(list)
    for n, data in grafo.nodes(data=True):
        # usa 0 como fallback caso 'semestre' não exista
        s = data.get('semestre', 0) or 0
        nodes_por_sem[s].append(n)
    semestres = sorted(nodes_por_sem.keys())
    if not semestres:
        semestres = [1]
    # Parâmetros de layout e aparência
    horizontal_spacing = 8.0
    vertical_spacing = 3.5
    box_width = 3.5
    box_height = 2.5
    # Calcular posições (x, y) para cada nó
    pos = {}
    for sem in semestres:
        # Ordenar os nós de forma alfabética
        nodes = sorted(nodes_por_sem[sem])
        x = sem * horizontal_spacing
        total = len(nodes)
        y_start = (total - 1) * vertical_spacing / 2
        for i, node in enumerate(nodes):
            y = y_start - i * vertical_spacing
            pos[node] = (x, y)

```

Figura 4.9: Função *desenhar_grafo_em_camadas*. Separados por comentários estão os blocos de código que agrupam os nós por semestres, os parâmetros utilizados e o cálculo de coordenadas (x, y) para cada nó. Fonte: Autora

Como mostrado na Figura 4.9, o agrupamento cria *nodes_por_sem* [nós] e ordena os semestres. Os parâmetros utilizados são *horizontal_spacing* que define a distância horizontal entre os centros das colunas: *vertical_spacing*, que define o espaço vertical entre nós dentro da mesma coluna, e os *box_width* e *box_height*, que definem o tamanho das caixas (nós). Para cada semestre, os nós são ordenados alfabeticamente e o código calcula as coordenadas (x, y) para cada um centralizando verticalmente os nós na coluna.

Para configurar a tela de desenho, é preciso criar uma figura vazia (toda a área de desenho) e um objeto (a área de plotagem específica dentro da figura onde o dígrafo será desenhado) como mostra a Figura 4.10.

```
# -----
# Preparar figura e eixos
# -----
# Largura da figura cresce com o número de semestre
largura = max(28, len(semestres) * 6.0)
# Altura é proporcional ao intervalo vertical total (y_max - y_min)
altura = max(12, (y_max - y_min) / 1.5)
fig = Figure(figsize=(largura, altura))
ax_grafo = fig.add_subplot(111)
```

Figura 4.10: Calcula largura e altura da figura com base no número de semestres e na faixa vertical; também cria a janela principal do desenho. Fonte: Autora

A biblioteca Matplotlib possibilita o uso de caixas personalizáveis, com *FancyBboxPatch*, como mostrado na Figura 4.11. Essas caixas são usadas para desenhar retângulos arredondados para as colunas do semestre em segundo plano e para as caixas das disciplinas (nós).

```
rect_coluna = FancyBboxPatch(
    (x_left, y_min),          # canto inferior esquerdo
    col_width, col_height,    # largura, altura
    boxstyle="round,pad=0.5", # estilo arredondado
    facecolor='#E2E8E8',      # cor de fundo (cinza claro)
    edgecolor='none',
    alpha=0.25,               # transparência
    zorder=0
)
ax_grafo.add_patch(rect_coluna)

# Título "Sem X"
ax_grafo.text(
    x_center, y_max - 0.4,
    f"Sem {sem}",
    ha='center', va='top',
    fontsize=13, fontweight='bold',
    zorder=4
)
```

Figura 4.11: Características estéticas das caixas de colunas dos semestres. Fonte: Autora

As arestas de pré-requisito são desenhadas usando *FancyArrowPatch* como mostrado na Figura 4.12. Os pontos iniciais e finais das seta são ajustados para se conectar precisamente às bordas das caixas, em vez de apontar para seus centros.

```

# -----
# Desenhando arestas orientadas
# -----
for start, end in grafo.edges():
    if start not in pos or end not in pos:
        continue
    s_pos = np.array(pos[start]) #Posição do nó inicial
    e_pos = np.array(pos[end]) #Posição do nó final
    vec = e_pos - s_pos          #Vetor direção e tamanho
    vec_len = np.linalg.norm(vec)
    if vec_len == 0:
        continue
    u_vec = vec / vec_len
    #Ajustar start_point e end_point para que tocar nas bordas
    start_point = s_pos.copy()
    end_point = e_pos.copy()
    if u_vec[0] > 0:
        start_point[0] += box_width / 2
        end_point[0] -= box_width / 2
    elif u_vec[0] < 0:
        start_point[0] -= box_width / 2
        end_point[0] += box_width / 2

    if u_vec[1] > 0:
        start_point[1] += box_height / 2
        end_point[1] -= box_height / 2
    elif u_vec[1] < 0:
        start_point[1] -= box_height / 2
        end_point[1] += box_height / 2

```

Figura 4.12: Para cada aresta $start \rightarrow end$, é calculado vetores entre centros (s_pos , e_pos), deslocando os pontos até as bordas das caixas. Fonte: Autora

No módulo de *viz.py* também configuramos a visualização dos mapas de calor. É utilizado o mapeamento de nota para cor, ou seja, uma cor específica que represente uma nota numérica. Como observado na função *map_nota_para_cor* na Figura 4.13, ela usa interpolação linear para criar um gradiente de cor suave.

```
def map_nota_para_cor(nota):  
    if nota >= 6.0:  
        t = (nota - 6.0) / 4.0  
        # Interpolação no cálculo do gradiente de cor  
        r = int(144 * (1 - t) + 0 * t)  
        g = int(238 * (1 - t) + 100 * t)  
        b = int(144 * (1 - t) + 0 * t)  
        return f"#{r:02x}{g:02x}{b:02x}"  
    else:  
        # Vermelho: 0.0 a 5.9  
        t = nota / 6.0  
        # gradiente da cor vermelha  
        r = int(139 * (1 - t) + 255 * t)  
        g = int(0 * (1 - t) + 182 * t)  
        b = int(0 * (1 - t) + 193 * t)  
        return f"#{r:02x}{g:02x}{b:02x}"
```

Figura 4.13: Mapeia uma nota (0-10) para uma cor no gradiente vermelho-verde. Nota 10 = Verde forte (#006400), nota 6 = Verde pastel fraco (#90EE90), nota 5.9 = Vermelho pastel fraco (#FFB6C1) e nota 0.0 = Vermelho forte (#8B0000). Fonte: Autora

Com o mapeamento das notas para cores, a função *desenhar_mini_mapa_calor* (Figura 4.14) ordena as notas em decrescente, crescente e por RA e desenha uma grade com até 5 colunas do mapa de calor junto com as células de cada nota usando *FancyBboxPatch*.


```
def desenhar_mini_mapa_calor(ax, x, y, width, height, notas, ordem='decrecente', ra_sequencia=None):
    """
    Parâmetros:
    - ordem: 'decrecente', 'crescente', 'ra'
    - ra_sequencia: lista ordenada de RAs para a ordem 'ra'
    """
    if not notas:
        return

    notas_validas = [n for n in notas if n is not None]
    quadrados_cinzas = [n for n in notas if n is None]

    notas_ordenadas = []

    # Ordenar as notas conforme a opção selecionada
    if ordem == 'decrecente':
        notas_ordenadas_validas = sorted(notas_validas, reverse=True)
        notas_ordenadas = notas_ordenadas_validas + quadrados_cinzas
    elif ordem == 'crescente':
        notas_ordenadas_validas = sorted(notas_validas)
        notas_ordenadas = notas_ordenadas_validas + quadrados_cinzas
    elif ordem == 'ra':
        # Na ordem 'ra', as notas já vêm na ordem correta (incluindo 'None')
        notas_ordenadas = notas
    else:
        # Padrão (decrecente)
        notas_ordenadas_validas = sorted(notas_validas, reverse=True)
        notas_ordenadas = notas_ordenadas_validas + quadrados_cinzas
```

Figura 4.14: Apenas as notas numéricas são filtradas nas *notas_validas*, é coletado apenas os None nos *quadrados_cinza* e é inicializado a lista vazia de *notas_ordenadas* que será montada nas diferentes opções de ordenação. Fonte: Autora

O desenho do mapa de calor é integrado na função *desenhar_grafo_em_camadas*. Quando as caixas dos nós são desenhadas, são reunidas todas as notas dos alunos para uma disciplina e, em seguida, o código faz o desenho do mini mapa de calor (Figura 4.15).

```

for node, (x, y) in pos.items():
    #Obter as notas pro mapa de calor
    disciplina = catalogo.get_disciplina(node)
    notas = []
    if disciplina and ra_sequencia:
        for ra in ra_sequencia:
            ultimo_registro = disciplina.get_ultima_nota_aluno(ra)
            if ultimo_registro and ultimo_registro['nota'] is not None and ultimo_registro['nota'] >= 0:
                notas.append(ultimo_registro['nota'])
            else:
                notas.append(None)
    rect = FancyBboxPatch(
        (x - box_width / 2, y - box_height / 2), # canto inferior esquerdo
        box_width, box_height, # largura, altura da caixa
        boxstyle="round,pad=0.1", # caixa arredondada
        facecolor='#FFFFFF',
        edgecolor='black',
        linewidth=1.3,
        zorder=3
    )
    # Interatividade de clique
    rect.set_picker(True) # a caixa principal clicável
    rect.set_gid(node) # Associa o ID do nó a ela
    ax_grafo.add_patch(rect)
    #Desenhar mini mapa de calor:
    mapa_height = box_height * 0.65
    desenhar_mini_mapa_calor(ax_grafo, x, y + box_height * 0.15, box_width * 0.9, mapa_height, notas,
                             ordem_heatmap, ra_sequencia)
    # Escreve o código da disciplina embaixo
    txt = ax_grafo.text(x, y - box_height * 0.4, node,
                        ha='center', va='center',
                        fontweight='bold', fontsize=8, zorder=5)

```

Figura 4.15: Para cada código de disciplina, o sistema itera por todos os RAs conhecidos para obter a última nota registrada de cada aluno, adicionada a uma lista e então passada para desenhar as células coloridas. Fonte: Autora

Com ajustes finais de leiaute, definição dos limites do gráfico, remoção dos eixos padrões, e adição de um fundo branco, o módulo *viz.py* retorna um objeto completo da classe.

4.5.3 Módulo *gui.py*

O módulo *gui.py* fornece a interface do usuário e a interatividade. Ele cria a janela maior com a exibição do grafo com seus nós, arestas, colunas e mapas de calor, um painel de informações detalhadas e os controles e interatividade com o mouse. A classe central do arquivo, *App*, é a responsável por todos esses componentes. O diagrama da classe *App* mostrado na Figura 4.16:

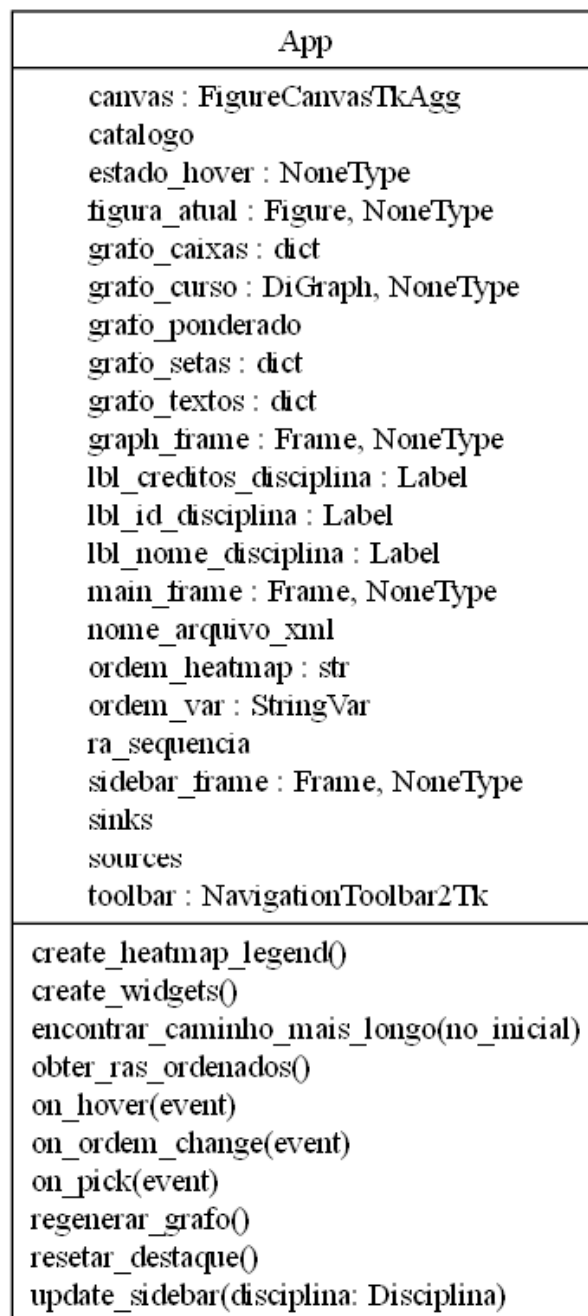


Figura 4.16: Diagrama da classe App. Fonte: Autora

Para construir o leiaute é usada a função *create_widgets* (Figura 4.17), responsável por organizar todas as partes da interface dentro da janela principal. Ele cria a barra lateral de informações (*sidebar_frame*) contendo: o nome do arquivo do currículo (arquivo XML), uma lista de estatísticas com a quantidade de disciplinas e alunos, um menu suspenso para escolher a ordenação do mapa de calor e um texto para mostrar a disciplina selecionada no clique de mouse.

```

def create_widgets(self):
    # Criar frame principal
    self.main_frame = ttk.Frame(self)
    self.main_frame.pack(side="top", fill="both", expand=True)

    # Barra lateral
    self.sidebar_frame = ttk.Frame(self.main_frame, width=220, padding=10, relief="sunken")
    self.sidebar_frame.pack(side="right", fill="y")
    self.sidebar_frame.pack_propagate(False)

    # Conteúdo da barra lateral
    ttk.Label(self.sidebar_frame, text="Informações", font=("Arial", 16, "bold")).pack(pady=5, anchor="w")
    ttk.Label(self.sidebar_frame, text=f"Currículo:", font=("Arial", 11, "bold")).pack(pady=(10, 0), anchor="w")
    ttk.Label(self.sidebar_frame, text=f"{self.nome_arquivo_xml}",
              foreground="darkblue", style='Italic.TLabel').pack(anchor="w")

    # Estatísticas
    print("GUI: Gerando grafo do catálogo...")
    self.grafo_curso = viz.criar_grafo_do_catalogo(self.catalogo)

```

Figura 4.17: Cria frames principais, cria a barra lateral, cria o título "Informações", prepara para escrever as estatísticas. Fonte: Autora

Para desenhar o gráfico e exibi-lo na janela, é usada a função *regenerar_grafo* (Figura 4.18), que é chamada para atualizar a exibição visual de acordo com a ordenação selecionada pelo usuário, criando um novo canvas do Matplotlib (*self.canvas*). A função usa a figura do módulo *viz.py* incorporada, chamando o *viz.desenhar_grafo_em_camadas*, descarta canvas antigos e cria *FigureCanvasTkAgg*, além de configurar os eventos de mouse.

```

def regenerar_grafo(self):
    """Regera o grafo com os parâmetros atuais de ordenação"""
    print(f"GUI: Regenerando grafo com ordem: {self.ordem_heatmap}")
    if not hasattr(self, 'graph_frame') or self.graph_frame is None:
        # Gerar nova figura com os parâmetros de ordenação
        self.figura_atual = viz.desenhar_grafo_em_camadas(
            self.grafo_curso,
            self.catalogo,
            self.ordem_heatmap,
            self.ra_sequencia)
        # Destruir canvas existente se houver
        if hasattr(self, 'canvas'):
            # Criar novo canvas
            self.canvas = FigureCanvasTkAgg(self.figura_atual, master=self.graph_frame)
            self.canvas.draw()
            self.toolbar = NavigationToolbar2Tk(self.canvas, self.graph_frame)
            self.toolbar.update()
            widget_canvas = self.canvas.get_tk_widget()
            widget_canvas.pack(side="top", fill="both", expand=True)
        # Recoletar referências dos elementos gráficos
        ax = self.figura_atual.axes[0]
        self.grafo_textos = {t.get_text(): t for t in ax.texts}
        self.grafo_setas = {}
        for artista in ax.get_children():
            self.grafo_caixas = {}
        for artista in ax.get_children():
            # Reconectar eventos
            self.canvas.mpl_connect('pick_event', self.on_pick)
            self.canvas.mpl_connect("motion_notify_event", self.on_hover)

```

Figura 4.18: O início da função usada para gerar e regenerar grafos. Fonte: Autora

A função `on_pick` extrai o código da disciplina clicada para solicitar ao Catalogo todas as informações (nome, código e créditos) que são exibidas na barra lateral com o `update_sidebar`.

A função `on_hover` destaca o nó, seus caminhos de seus pré-requisitos e todas as disciplinas que dependem dela alterando a aparência das bordas dos nós e cor das arestas. O código dentro da função usado para mudar a estética ao passar o mouse é mostrado na Figura 4.19.

```

if hovered_label:
    if self.estado_hover == hovered_label:
        return
    self.estado_hover = hovered_label
    connected_nodes = self.encontrar_caminho_mais_longo(hovered_label)
    # Destacar textos
    for label, text_obj in self.grafo_textos.items():
        if label == hovered_label:
            text_obj.set_color("blue")
            text_obj.set_fontsize(10)
        elif label in connected_nodes:
            text_obj.set_color("blue")
        else:
            text_obj.set_color("black")
            text_obj.set_fontweight("normal")
            text_obj.set_fontsize(9)

    # Destacar caixas
    for gid, caixa in self.grafo_caixas.items():
        if gid == hovered_label:
            caixa.set_edgecolor("blue")
            caixa.set_linewidth(2.5)
        elif gid in connected_nodes:
            caixa.set_edgecolor("blue")
            caixa.set_linewidth(2.0)
        else:
            caixa.set_edgecolor("black")
            caixa.set_linewidth(1.3)

```

Figura 4.19: Destacar caixas (nós) e textos ao passar do mouse. Fonte: Autora

Assim que a função verifica que o mouse está em cima da caixa de disciplina, é chamada a função *encontrar_caminhos_mais_longo* para buscar todos os pré-requisitos, todas as dependências dela e a disciplina em questão, usando funcionalidades que a biblioteca NetworkX proporciona, como a *nx.ancestors* para antecessores do nó e *nx.descendants* para sucessores do nó. Depois são atualizadas as cores do texto dentro do nó, das bordas e das arestas em azul para se destacar do preto e branco dos outros nós e arestas.

4.5.4 Main.py

O arquivo *main.py* é o que deve ser executado. Nele a função *selecionar_arquivos* abre duas janelas para o usuário escolher o arquivo XML da estrutura curricular e para escolher o arquivo CSV dos históricos dos alunos.

Na função *main*, depois da seleção de arquivos a classe *Catalogo* faz a leitura e estruturação dos dados com a função *carregar_dados*. Ao final a classe *App* é chamada para criar a janela principal e o *mainloop()* mantém a janela aberta e escuta eventos do usuário. Assim que a janela é fechada, o programa é encerrado.

4.6 Proposta de Visualização

Os arquivos usados para a primeira visualização foram o currículo do curso de Sistemas da Informação (curso 96) de 2020 e a simulação do GradeGen, contendo:

- 10 alunos abaixo da média (nota entre 0 e 5);
- 10 alunos na média (nota entre 5 e 7);
- 10 alunos acima da média (nota entre 7 e 10).

As disciplinas sorteadas pelo GradeGen para sofrer mudanças abruptas de impacto:

- SI200 - Impacto negativo (disciplina difícil)
- TT106 - Impacto positivo (disciplina fácil)
- SI916 - Impacto negativo (disciplina difícil)
- SI912 - Impacto negativo (disciplina difícil)
- SI304 - Impacto positivo (disciplina fácil)
- SI450 - Impacto positivo (disciplina fácil)

A visualização renderizada com esses arquivos de entrada foi a da Figura 4.20, com a área maior do LDAGs e a barra lateral de informações e controle à direita.

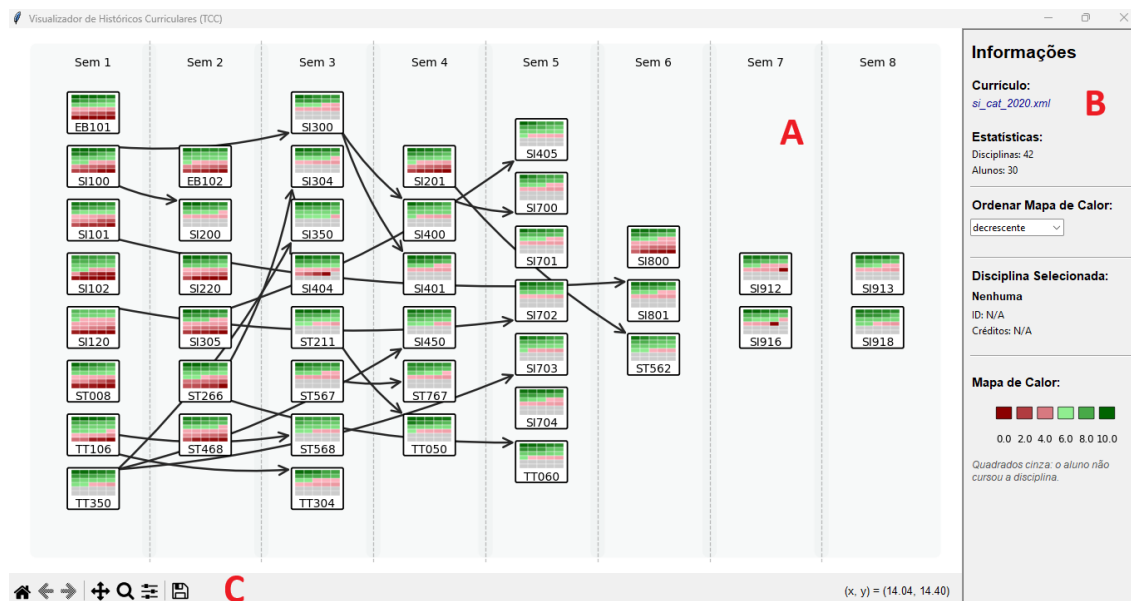


Figura 4.20: Sistema renderizado na primeira simulação com o currículo de Sistemas de Informação 2020 e 30 alunos. A seção A representa a área do grafo, a seção B representa a barra de informações e a Seção C é a barra de ferramentas de navegação. Fonte: Autora

Na seção A da Figura 4.20, cada coluna vertical está rotulada na parte superior com o número de cada semestre (Ex.: Sem 2, Sem 3, etc) e o grafo está disposto em um leiaute horizontal, similar a uma linha do tempo, trazendo uma progressão temporal e adaptado para o fluxo de leitura ocidental, da esquerda para a direita. Os nós estão identificados pelos códigos das disciplinas na parte inferior dos retângulos arredondados. Os mapas de calor preenchem a maior parte do centro de cada nó e as arestas possuem um roteamento simples, que colocam seu ponto inicial no canto da borda do nó que está mais próximo do nó de destino e seu ponto final no canto da borda do nó que está mais próximo do nó de origem, visando minimizar o comprimento das arestas. Quando os nós estão no mesmo nível de altura a aresta tem seu ponto inicial fica na metade da altura da borda e seu ponto final na mesma localização do nó de destino.

Ao passar o mouse sobre uma disciplina, o sistema destaca todas as disciplinas que são pré-requisito dela (direta ou indiretamente) e todas as disciplinas dos quais ela é pré-requisito (direta ou indiretamente), como mostrado na Figura 4.21.

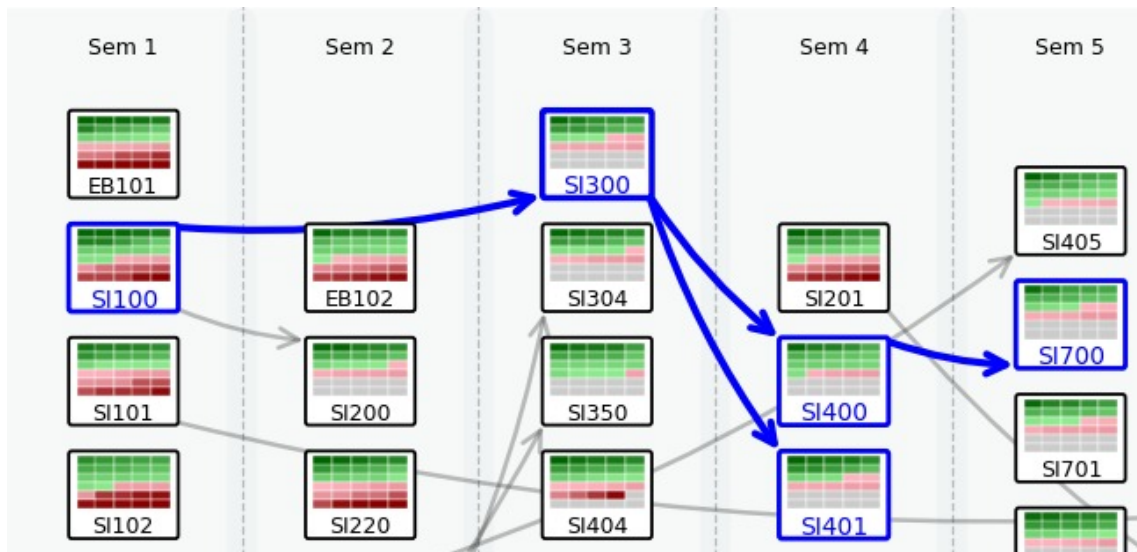


Figura 4.21: O cursor está em cima do nó da disciplina SI300, destacando em azul escuro as arestas originadas no seu nó antecessor SI100 e as dependentes dela SI400, SI401 e SI700. Fonte: Autora

Em cada nó, um grid com um mini mapa de calor é apresentado com uma largura fixada em 5 e adaptado para o número de alunos (nesse caso da simulação são 30 alunos).

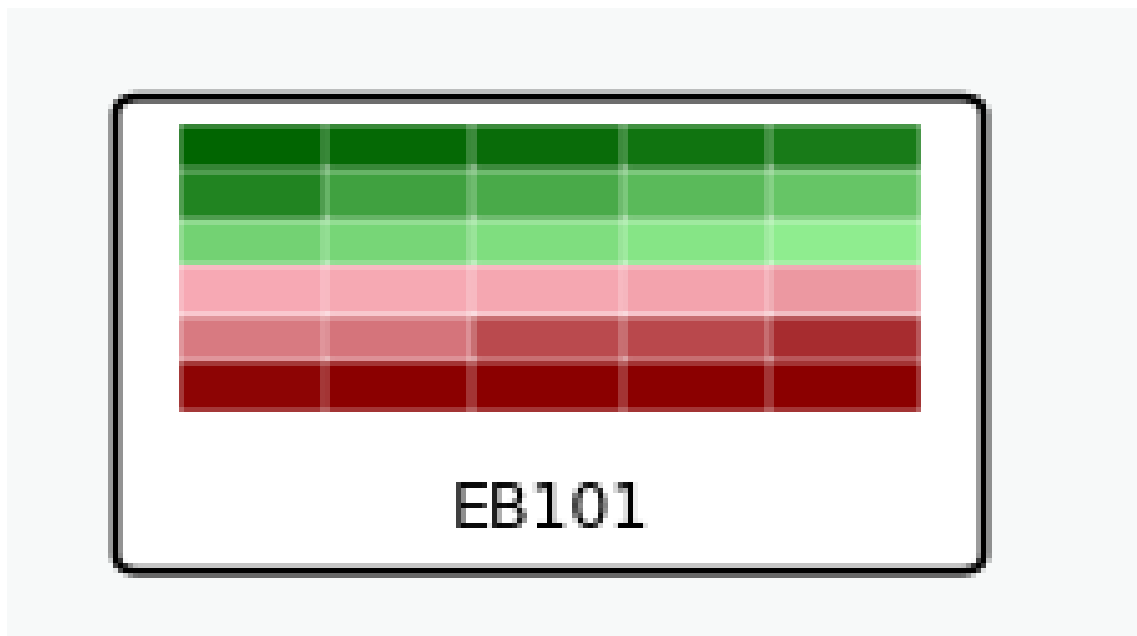


Figura 4.22: Zoom in no nó da disciplina EB101 com um mapa de calor. Fonte: Autora

A Figura 4.22 mostra 30 notas de 30 alunos, sendo que 15 tiveram uma nota maior ou igual a 6.0 e os outros 15 tiveram nota menor que 6.0. Vale notar que a nota mostrada no mapa de calor é a última nota registrada para o aluno na disciplina, ou seja, se um aluno cursou pela

primeira vez a disciplina EB101 e obteve a nota 4.8 e depois cursou ela de novo e dessa vez obteve 6.7 no mapa de calor, a nota representada é a 6.7. Isso ajuda a entender quais são os alunos que ainda estão atrasados e "travados" em disciplinas.

Na barra lateral à direita (seção B da Figura 4.20) se encontram as informações gerais do sistema como: nome do arquivo usado para extrair o currículo do curso; as estatísticas, incluindo a quantidade de disciplinas e a quantidade de alunos na simulação; uma barra de seleção para diferentes tipos de ordenação do mapa de calor: decrescente (*default* do sistema), crescente e por RA (ou seja, as posições de cada quadrado em todos os nós representam um mesmo aluno); uma área para a disciplina selecionada pelo clique de mouse e a legenda do mapa de calor. A barra inicial do sistema, assim que a visualização é gerada, se assemelha à da Figura 4.23.

Informações

Currículo:
[si_cat_2020.xml](#)

Estatísticas:
Disciplinas: 42
Alunos: 30

Ordenar Mapa de Calor:
decrecente ▾

Disciplina Selecionada:
Nenhuma
ID: N/A
Créditos: N/A

Mapa de Calor:

0.0 2.0 4.0 6.0 8.0 10.0

Quadrados cinza: o aluno não cursou a disciplina.

Figura 4.23: A barra lateral gerada na simulação. O nome do arquivo do currículo de SI está como *si_cat_2020.xml*. A ordenação está em *default* e abaixo, como o mouse não clicou em nenhum nó, a disciplina selecionada está como "Nenhuma". Fonte: Autora

Assim que o usuário clica em um nó específico, as informações da disciplina aparecem na barra lateral como na Figura 4.24:

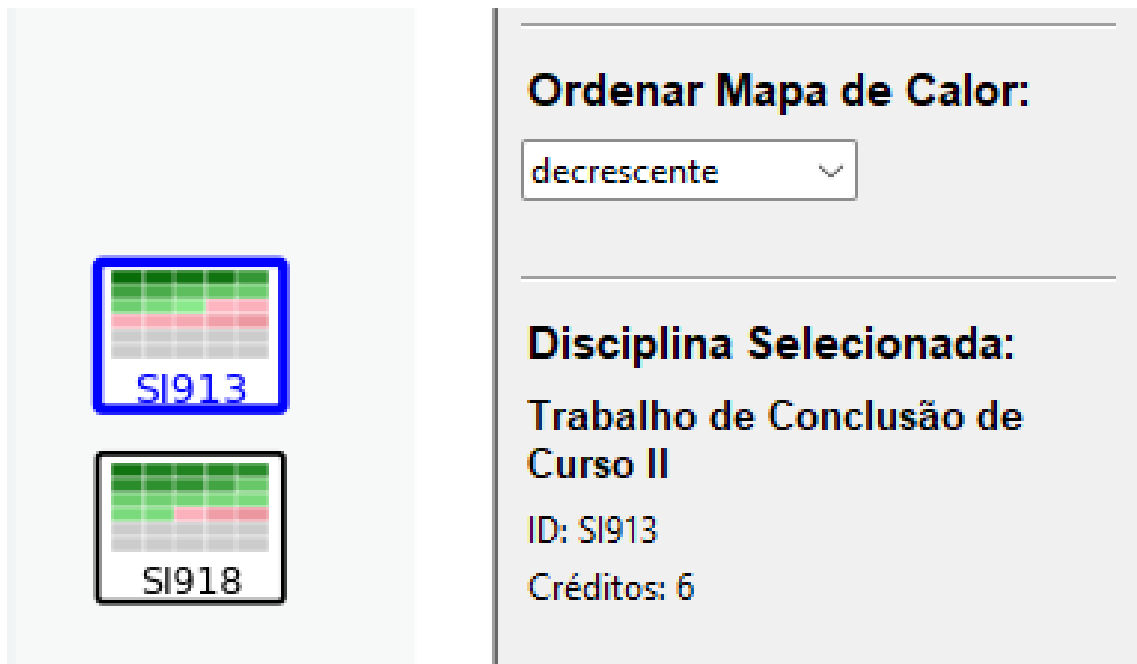


Figura 4.24: O nó da disciplina SI913 foi clicado, as informações sobre ela são preenchidas na área de "Disciplina Selecionada" com o nome por extenso, o ID (código) e os créditos. Fonte: Autora

O sistema também incorpora uma barra de ferramentas de navegação (seção C da Figura 4.20), a *NavigationToolbar2Tk*, um módulo da *backend_tkagg* que fornece recursos interativos para gráficos do Matplotlib incorporados em uma interface gráfica Tkinter. Os botões padrões nela são:

- Botão Home: restaura a visão original e padrão do gráfico;
- Botão Voltar: funciona como um "undo" da navegação;
- Botão Avançar: funciona como um "redo" da navegação;
- Botão Pan: mover o gráfico com o arrastar do mouse;
- Botão Zoom: Ampliar áreas específicas do gráfico;
- Botão Subplots: Usado para configurar dimensões do gráfico;
- Botão Salvar: Para exportar a figura do gráfico como PNG, PDF, SVG, etc.

No canto direito são mostradas as coordenadas (x, y) da posição atual do mouse, uma funcionalidade padrão da *NavigationToolbar2Tk*.

Para mais testes, foi renderizado um LDAG com a estrutura curricular do curso de Engenharia Ambiental (curso 89) do catálogo de 2023. Uma simulação foi criada com ele no GradeGen com os parâmetros de alunos:

- 15 alunos abaixo da média (nota entre 0 e 5);
- 15 alunos na média (nota entre 5 e 7);
- 10 alunos acima da média (nota entre 7 e 10).

As disciplinas a seguir foram configuradas em disciplinas de baixa dificuldade (acréscimo na nota da turma): EB306; EB801; EB902 e EB904. Foram definidas como disciplinas com alta dificuldade (decréscimo na nota da turma): EB201; EB305; EB403 e EB504.

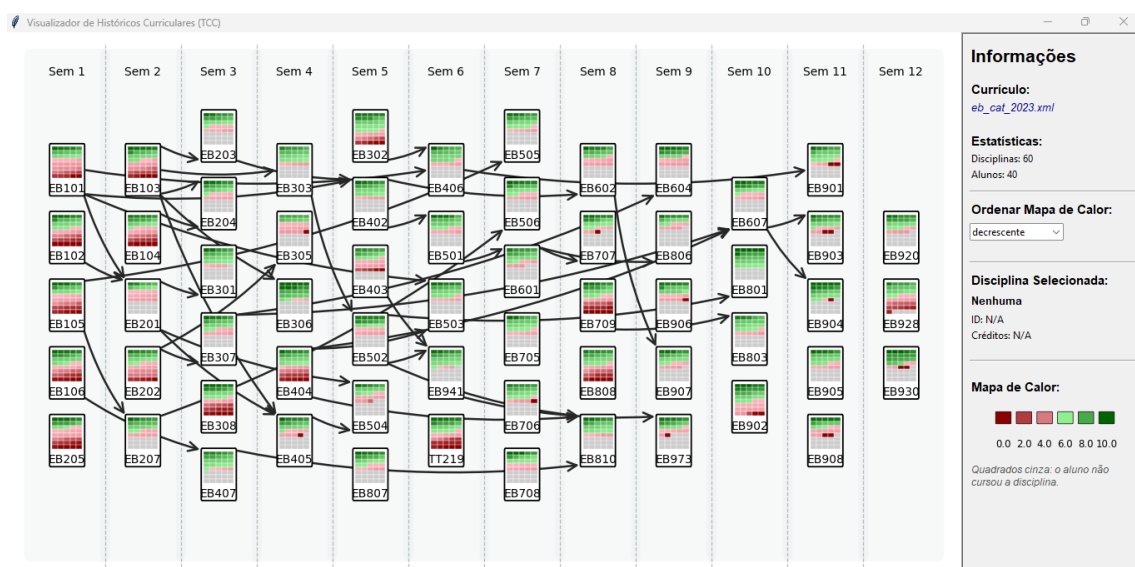


Figura 4.25: Visualização do curso de Engenharia Ambiental de 2023, com maior quantidade de alunos, disciplinas e semestres. Fonte: Autora

A Figura 4.25 mostra as caixas dos nós se adaptando ao tamanho do grafo. Como o currículo tem mais semestres do que o de Sistemas de Informação, os nós estão menos largos. O mapa de calor ficou com uma altura maior devido a quantidade de alunos (40 alunos nessa simulação) e com as células proporcionais ao seu tamanho.

Capítulo 5

Resultados

Os resultados proporcionados pela visualização se relacionam diretamente com o levantamento de requisitos e os trabalhos revisados. Tomando a simulação do curso de Engenharia Ambiental de 2023 da Figura 4.25 com 40 alunos e 60 disciplinas distribuídas em 12 semestres.

É possível identificar, com a ajuda da visualização, os gargalos do curso, disciplinas nos primeiros semestres que funcionam como barreiras para o fluxo dos semestres subsequentes.



Figura 5.1: Evidência de um gargalo no curso de Engenharia Ambiental Fonte: Autora

Na Figura 5.1 as disciplinas EB101 e EB102 do 1º semestre apresentam uma taxa de reprovação alarmante (grande maioria das células vermelhas) e servem como pré-requisito de EB201 no 2º semestre, que apresenta muitas células cinzas; portanto, muitos alunos vão atrasar a sua graduação já no 2º semestre e a disciplina EB201 vai precisar de mais vagas no próximo ano para acomodar esses atrasos.

A grande quantidade de células cinzas evidencia como muitos alunos vão sofrer com atraso na sua integralização. De uma turma de 40 alunos, poucos chegaram ao sexto ano do curso com um coeficiente de progressão perto de 1,0 já que ainda possuem muitas pendências em disciplinas do decorrer do curso. Isso satisfaz o requisito de visualizar padrões e pendências de alunos e de identificar necessidades de vagas em disciplinas mencionados na Seção 4.1.

Os parâmetros da simulação do GradeGen e dos pré-requisitos nas disciplinas são importantes para a análise. Por exemplo, a disciplina EB201, classificada como difícil, tem apenas 10 notas acima de 6.0 e 15 notas abaixo de 6.0, além de possuir 15 células em cinza (os

alunos que não foram aprovados nos seus pré-requisitos, a EB101 e EB102), ou seja, seu mapa de calor é majoritariamente vermelho e cinza. A disciplina EB902, classificada como fácil, possui 24 alunos com nota maior ou igual a 6,0, apresentando um mapa de calor em sua maioria verde. Assim, a análise comparativa entre as disciplinas, notas e históricos ajuda a compreender as dificuldades na perspectiva dos alunos.

O destaque do caminho entre os nós auxilia a visualizar o tamanho dos caminhos de integralização entre um nó e outro. Por exemplo, a disciplina EB901 (Figura 5.2) que deve ser cursada idealmente no 11º semestre tem como pré-requisito a disciplina EB406, cursada apenas no 6º semestre. A EB406 também tem como um dos pré-requisitos a EB101, cursada no 1º semestre; isso evidencia como o caminho entre essas disciplinas é bastante longo, aumentando a probabilidade de os alunos esquecerem os conteúdos aprendidos nelas e não associarem os conceitos entre elas.

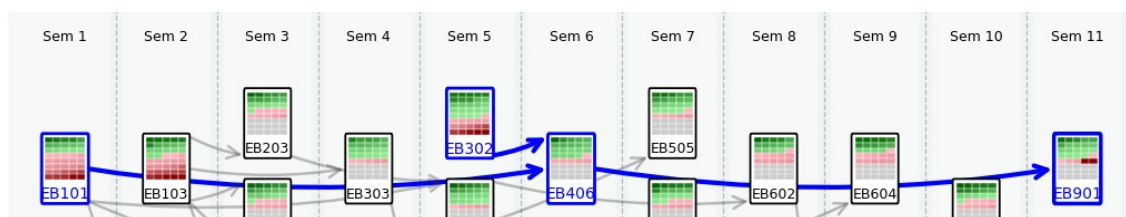


Figura 5.2: Cursor em cima da disciplina EB901, destacando o trajeto que é preciso cursar para chegar até ela. Fonte: Autora

Ao mesmo tempo, muitas disciplinas tem uma alta centralidade, dependendo de muitas disciplinas e atuando como barreira no avanço do curso em seu caminho aumentando a complexidade e densidade do currículo. Como a disciplina EB402 da Figura 5.3.

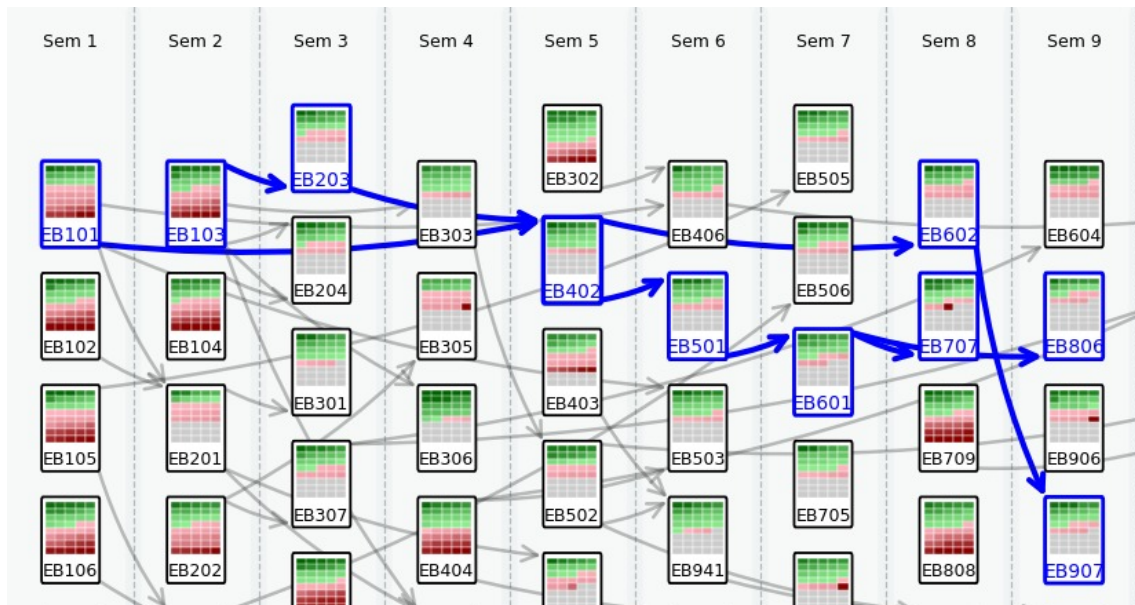


Figura 5.3: A disciplina EB402 depende direta e indiretamente de 3 disciplinas (EB101, EB103 e EB203) e ela serve como barreira para 6 disciplinas (EB501, EB602, EB707, EB601, EB806 e EB907). Fonte: Autora

Esses conceitos são identificáveis em outros catálogos. A simulação gerada agora foi do curso de Sistemas de Informação (curso 96) do catálogo de 2026 com os parâmetros de alunos:

- 10 alunos abaixo da média (nota entre 0 e 5);
- 15 alunos na média (nota entre 5 e 7);
- 10 alunos acima da média (nota entre 7 e 10).

Para os parâmetros de disciplinas de baixa dificuldade são: SI100; SI250 e SI305; SI702. E as disciplinas de alta dificuldade são: EB101; EB102; SI201 e SI601.

A visualização gerada pelo sistema na ordenação de RA ficou como a da Figura 5.4:

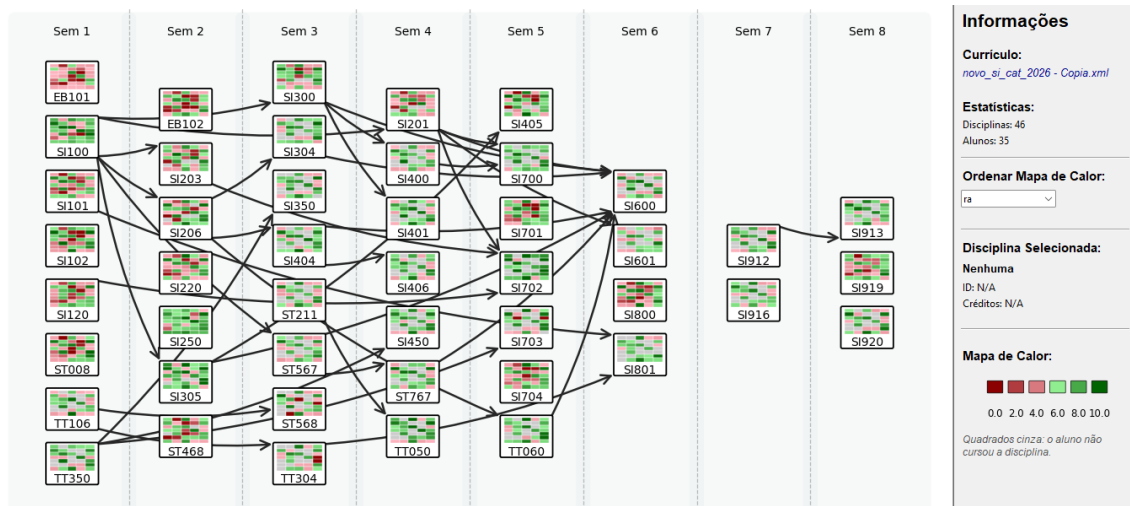


Figura 5.4: Curso de Sistemas de Informação de 2026 com os mapas de calor no ordenação por RA. Fonte: Autora

Nessa ordenação de RA, todas as células das disciplinas seguem a mesma ordem de RA, ou seja, a n -ésima célula do mapa de calor da SI100 representa o mesmo aluno da n -ésima célula da SI203.

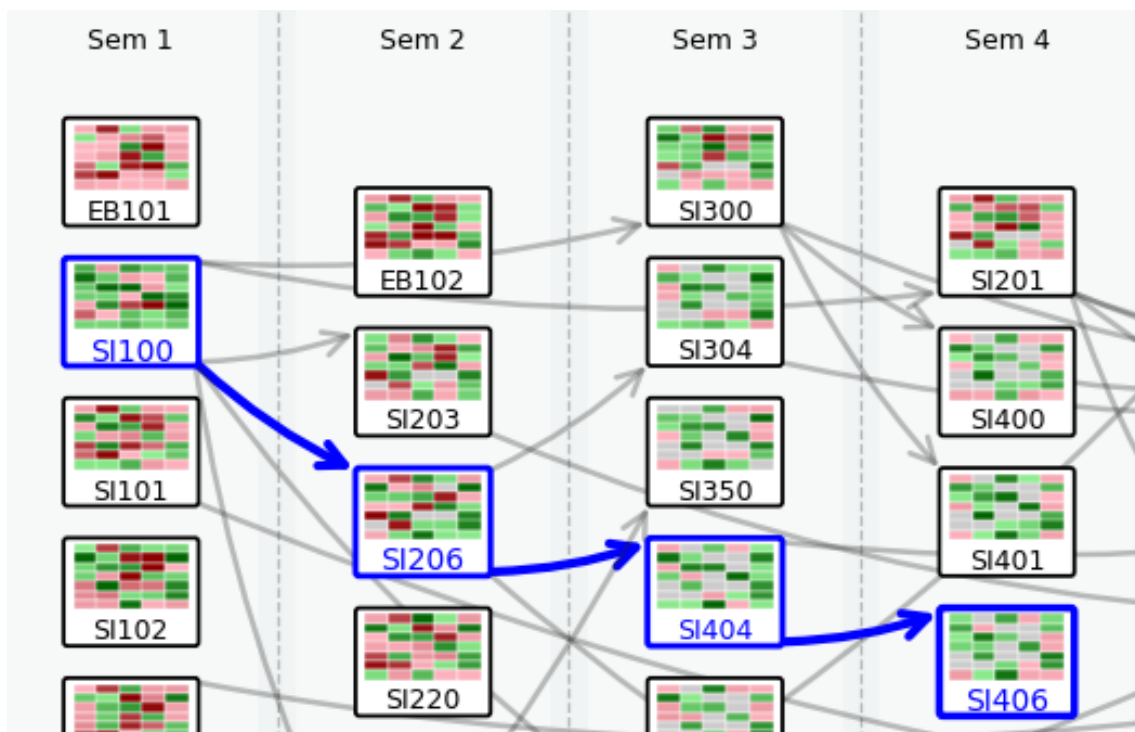


Figura 5.5: Caminho da SI100 destacado. Fonte: Autora

Na Figura 5.5, é possível perceber na segunda célula da primeira linha das disciplinas em destaque um aluno que obteve uma nota ruim na SI100, obteve uma nota pior na SI206 e portanto não cursou a SI404 e SI406.

Sendo assim, com todas as análises provenientes do LDAG, fica claro que a ferramenta tem grande potencial de fornecer melhorias consideráveis no trabalho de coordenadores de curso. Pode prover informações para análises e ajudar em tomadas de decisão, como priorizar monitores para disciplinas que possuem um desempenho acadêmico baixo, identificar riscos de desistências e trancamentos com alunos que reprovaram em disciplinas iniciais, identificar as disciplinas gargalo que atuam como barreira no fluxo acadêmico e fundamentar reformulações curriculares (analisando caminhos longos de pré-requisito, semestres densos e tempo de integralização realista).

Alguns pontos de melhoria podem ser analisados depois da construção desse protótipo:

- Melhoria no roteamento de arestas: Em um currículo com uma grande quantidade de pré-requisitos, as arestas podem causar uma desordem visual (*visual clutter*);
- Destacar trajetórias individuais: O mapa de calor pode dificultar a visão de um único aluno, então visualizar um único RA pode ajudar a tratar casos específicos de alunos;
- Implementação das métricas de grafos: O cálculo de medidas como grau ponderado de nós, centralidade e complexidade é relevante tanto para os coordenadores terem dados sólidos nas reformulações curriculares quanto para os alunos entenderem mais sobre suas vidas acadêmicas;
- Aperfeiçoamento da estética do leiaute: Os tamanhos dos nós e mapas de calor podem ficar desproporcionais em currículos com muitos semestres.

Os resultados obtidos evidenciam o potencial do sistema baseado em LDAGs como uma ferramenta de apoio à gestão acadêmica e à visualização de currículos e históricos. Embora o protótipo ainda possa ser aprimorado em aspectos visuais e funcionais, essa abordagem pode contribuir significativamente para decisões pedagógicas mais embasadas, promovendo currículos mais integrados e alinhados à realidade acadêmica dos alunos.

O código completo para o sistema está disponibilizado no GitHub (SISTEMA..., 2025).

Capítulo 6

Conclusões

A proposta de visualização deste trabalho buscou superar as limitações de sistemas de gestões já existentes, ajudar coordenadores e professores a entenderem melhor o currículo do curso e como são os padrões de desempenho acadêmico dos alunos, com a exibição de diversas trajetórias acadêmicas e implementar técnicas de visualizações diferentes em um único sistema.

A análise bibliográfica e de trabalhos relevantes para o tema como o sistema GradeGen, o CourseViewer e o software Curricular Analytics fundamentaram a escolha das técnicas de visualização e a integração entre a estrutura de grafos, matrizes e mapas de calor como as bases da visualização de dados educacionais escolhidas.

O sistema desenvolvido demonstrou que a combinação entre LDAGs e mapas de calor podem gerar gráficos que mantêm a coerência da estrutura curricular e ao mesmo tempo, mostram padrões de desempenho acadêmico e gargalos de progressão.

Os resultados obtidos possibilitaram uma análise mais rica, como a identificação de disciplinas com altos índices de reprovação, necessidade de mais vagas em disciplinas específicas para os próximos anos, trajetórias de alunos com dificuldades e caminhos longos de pré-requisitos que podem afetar a integralização do curso. Isso pode otimizar o trabalho das coordenações de curso, permitindo decisões pedagógicas mais informadas. Além disso, a implementação de interações dinâmicas contribuiu para tornar o processo de análise mais intuitivo e eficiente.

Conclui-se, portanto, que o protótipo desenvolvido constitui uma abordagem promissora para tornar a gestão acadêmica mais analítica e visual.

Como perspectiva futura, sugere-se aprimoramentos de roteamento de arestas, refinamento estético do leiaute e cálculos automáticos de centralidade e complexidade, junto a uma integração a um banco de dados institucional real. Dessa forma, a solução poderá evoluir de um protótipo para uma plataforma real de apoio à gestão, ajudando efetivamente a melhoria contínua dos currículos e do acompanhamento estudantil.

Referências bibliográficas

Aldrich, P. R. The curriculum prerequisite network: Modeling the curriculum as a complex system. en. **Biochem. Mol. Biol. Educ.**, Wiley, v. 43, n. 3, p. 168–180, mai. 2015. DOI: 10 . 1002/bmb . 20861.

Barros, R.; Mendes, T.; Silva, C. da. Use of reorderable matrices and heatmaps to support data analysis of students transcripts. In: ANAIS Estendidos da XXXII Conference on Graphics, Patterns and Images. Rio de Janeiro: SBC, 2019. P. 219–222. DOI: 10 . 5753 / sibgrapi . est . 2019 . 8334. Disponível em: <[https : //sol.sbc.org.br/index.php/sibgrapi_estendido/article/view/8334](https://sol.sbc.org.br/index.php/sibgrapi_estendido/article/view/8334)>.

Card, S. K.; Mackinlay, J. D.; Shneiderman, B. **Readings in information visualization - using vision to think**. [S.l.]: Academic Press, 1999. ISBN 978-1-55860-533-6.

CURRICULAR Analytics. 2025. Disponível em: <<https://curricularanalytics.org/home>>. Acesso em: 14 mai. 2025.

Ellis, G.; Dix, A. An explorative analysis of user evaluation studies in information visualisation. In: PROCEEDINGS of the 2006 AVI Workshop on BEyond Time and Errors: Novel Evaluation Methods for Information Visualization. Venice, Italy: Association for Computing Machinery, 2006. (BELIV '06), p. 1–7. ISBN 1595935622. DOI: 10 . 1145/1168149 . 1168152. Disponível em: <<https://doi.org/10.1145/1168149.1168152>>.

Free, H. (Ed.). **GitHub - Curricular Analytics**. 2024. Disponível em: <<https://github.com/CurricularAnalytics/CurricularAnalytics.jl>>. Acesso em: 12 jun. 2025.

FUTURE Trends Forum. 2021. Disponível em: <[https : //youtu.be/Ou5h0tg-5q4](https://youtu.be/Ou5h0tg-5q4)>. Acesso em: 10 jun. 2025.

IBM. **O que é DAG (directed acyclic graph)? | IBM**. 2025. Disponível em: <<https://www.ibm.com/br-pt/think/topics/directed-acyclic-graph>>. Acesso em: 27 set. 2025.

Mohseni, Z.; Masiello, I.; Martins, R. M.; Nordmark, S. Visual Learning Analytics for Educational Interventions in Primary and Secondary Schools: A Scoping Review. **Journal of Learning Analytics**, v. 11, n. 2, p. 91–111, jun. 2024. DOI: 10 . 18608/jla . 2024 . 8309. Disponível em: <<https://learning-analytics.info/index.php/JLA/article/view/8309>>.

Morroni, J. A. et al. Defining Information Visualization heuristics based on heuristic grouping by experts. PrePrint. [S.l.], 2023.

Munzner, T. **Visualization Analysis and Design**. [S.l.]: A K Peters/CRC Press, dez. 2014.

Oliveira, G. A. de. **GradeGen-Geração e adaptação de dados para sistemas de mineração de dados educacionais**. 2020. Monografia (Trabalho de Conclusão de Curso) – Universidade Estadual de Campinas/FT, Limeira.

Palomo, G. M. **Melhoria de leiaute de grafos acíclicos direcionados em camadas no software CourseViewer**. 2019. Monografia (Trabalho de Conclusão de Curso) – Universidade Estadual de Campinas/FT, Limeira.

Praça, O. P. **Avaliação de usabilidade e de aspectos de visualização de informação no software CourseViewer**. 2019. Monografia (Trabalho de Conclusão de Curso) – Universidade Estadual de Campinas/FT, Limeira.

Raji, M. et al. Visual progression analysis of student records data. In: 2017 IEEE Visualization in Data Science (VDS). Phoenix, AZ: IEEE, out. 2017.

Romero, C.; Ventura, S. Educational Data Mining: A Review of the State of the Art. **IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)**, v. 40, n. 6, p. 601–618, 2010. DOI: 10.1109/TSMCC.2010.2053532.

Schwendimann, B. A. et al. Perceiving Learning at a Glance: A Systematic Literature Review of Learning Dashboard Research. **IEEE Transactions on Learning Technologies**, v. 10, n. 1, p. 30–41, 2017. DOI: 10.1109/TLT.2016.2599522.

Shneiderman, B. The eyes have it: a task by data type taxonomy for information visualizations. In: PROCEEDINGS 1996 IEEE Symposium on Visual Languages. [S.l.: s.n.], 1996. P. 336–343. DOI: 10.1109/VL.1996.545307.

Silva, C. da; Inoue, M.; Mendonça, P. de. CourseViewer – Ferramenta para visualização de catálogos de cursos universitários e históricos escolares. In: ANAIS do VIII Simpósio Brasileiro de Sistemas de Informação. São Paulo: SBC, 2012. P. 341–352. DOI: 10.5753/sbsi.2012.14418. Disponível em: <<https://sol.sbc.org.br/index.php/sbsi/article/view/14418>>.

SISTEMA de Visualizações de Múltiplos Históricos Escolares. 2025. Disponível em: <<https://github.com/maryeduardasilva/Curricular-Viewer>>. Acesso em: 12 nov. 2025.

Sugiyama, K.; Tagawa, S.; Toda, M. Methods for Visual Understanding of Hierarchical System Structures. **IEEE Transactions on Systems, Man, and Cybernetics**, v. 11, n. 2, p. 109–125, 1981. DOI: 10.1109/TSMC.1981.4308636.

UC San Diego - Curricular Analytics. 2025. Disponível em: <<https://educationalinnovation.ucsd.edu/curricular-analytics/index.html>>. Acesso em: 14 mai. 2025.